UNIVERSITY
OF OSLO

Master's thesis

# Rethinking Multilevel K-Way Graph Partitioning using an Objective Function that Considers the Distribution of Communication volume

**Riccardo Ieva**

Informatics: Programming and Systems Architecture

Department of Informatics
Faculty of Mathematics and Natural Sciences

Spring 2026

**Riccardo Ieva**

# Rethinking Multilevel K-Way Graph Partitioning using an Objective Function that Considers the Distribution of Communication volume

Supervisors:
Xing Cai
James D. Trotter

# Disclaimer

In this scientific work, generative artificial intelligence (AI) has been used. All data and personal information have been processed in accordance with the University of Oslo's regulations, and I, as the author of the document, take full responsibility for its content, claims, and references. An overview of the use of generative AI is provided below.

The service Claude, developed by Anthropic, has been used to improve the content of the thesis. A first version of the work was pasted in its entirety, and the model was given the prompt "Fix my grammar in this master thesis". The text was then iterated a few times through the model where new prompts were used to get the correct structure of the text. The final result was partially cut out, fact-checked, and mostly rewritten by the author.

**Abstract**

Here come 3–6 sentences describing your thesis.

# Preface

Here comes your preface, including acknowledgments and thanks.

# Contents


1 Introduction . . . 1
1.1 Thesis Goal . . . 2
1.2 Thesis Structure . . . 2
2 Background . . . 5
2.1 Graph Partitioning Problem . . . 5
2.1.1 Objective Functions . . . 6
2.1.2 Shortcomings of the Objective Functions . . . 6
2.2 Graph Partitioning Algorithms . . . 7
2.2.1 Recursive Bisection . . . 7
2.2.2 Spectral Bisection . . . 8
2.2.3 Graph Growing Partitioning . . . 8
2.2.4 Streaming Graph Partitioning . . . 8
2.2.5 Multilevel Graph Partitioning . . . 8
2.3 Applications of Graph Partitioning . . . 9
2.3.1 Parallel Processing . . . 9
2.3.2 Complex Networks . . . 9
2.3.3 VLSI Physical Design . . . 10
2.3.4 Neural Networks . . . 10
3 Multilevel K-Way Graph Partitioning . . . 11
3.1 Preliminaries . . . 11
3.2 Coarsening Phase . . . 11
3.2.1 Sorted Heavy Edge Matching . . . 13
3.3 Initial Partitioning Phase . . . 13
3.4 Uncoarsening Phase . . . 14
3.4.1 Projection and preparation before refinement . . . 14
3.4.2 Greedy Refinement . . . 15
4 Communication Volume Calculation . . . 19
4.1 Basic definitions . . . 20
4.2 Total gain calculation . . . 21
4.3 Partition specific gain calculation . . . 22
4.4 Examples . . . 23
4.4.1 Two Partition example . . . 24
4.4.2 Three partition example . . . 25
4.4.3 Case 4 example . . . 27
4.5 Implementation . . . 28
4.5.1 Total gain implementation . . . 29
4.5.2 Partition specific gain implementation . . . 30

5 Refinement Using Communication Volume . . . 31
5.1 Refinement using total communication volume . . . 31
5.2 Refinement using partition-specific communication volume . . . 32
5.2.1 Move selection . . . 32
5.2.2 Encouraging more moves . . . 33
5.2.3 Implementation of refinement . . . 33
6 METIS and Supporting Tooling . . . 37
6.1 METIS . . . 37
6.1.1 Implementing NVOL . . . 37
6.1.2 Inputs . . . 39
6.1.3 Output . . . 39
6.2 Graph Tools . . . 40
6.2.1 calc_partitions . . . 40
6.2.2 form_statics . . . 40
6.2.3 graph_results_vol . . . 41
6.2.4 stats_vol . . . 41
6.2.5 translate_to_metis . . . 41
6.2.6 model_estimates . . . 41
7 Results . . . 43
7.1 Graphs used for testing . . . 43
7.2 Maximum outgoing volume per partition . . . 44
7.2.1 Heart dataset . . . 45
7.2.2 SuiteSparse Matrix Collection . . . 48
7.3 Tuning NVOL . . . 58
7.3.1 Heart dataset with 4% imbalance . . . 58
7.3.2 SuiteSparse Matrix Collection with 4% imbalance . . . 59
7.4 Model estimates . . . 61
7.4.1 coPapersDBLP graph estimate results . . . 63
7.4.2 italy_osm graph estimate results . . . 63
8 Future Work . . . 67
8.1 Improved efficiency . . . 67
8.2 Introduce hardware knowledge . . . 68
8.3 More testing . . . 68
9 Conclusions . . . 69

# List of Figures


2.1 Vertex $A$ has an outgoing communication volume of 1 but an edge-cut of 2. Each color represents a partition. . . . . 7

3.1 The phases of the multilevel $k$-way partitioning algorithm. During the coarsening phase, the vertices are contracted together and the graph size is successively decreased; during the initial partitioning phase, a $k$-way partition is performed on the smallest graph; and during the uncoarsening phase, the partitions are successively refined and projected back to a larger graph. . . 12
3.2 The process of bisecting a graph into 7 partitions.. . . . . 13

4.1 Vertex $A$ has an outgoing communication volume of 2. Each color represents a partition. . . . . 20
4.2 Vertex $A$ has an outgoing communication volume of 1. Each color represents a partition. . . . . 20
4.3 The total communication volume is 8. . . . . 21
4.4 Partitioned graph with a volume of 4, each partition has an outgoing volume of 2. . . . . 24
4.5 Partitinoed graph with a volume of 4, each partition has an outgoing volume of 2 after the move. . . . . 24
4.6 Partitioned graph with a total communication volume of 8.. . . . . 25
4.7 Partitioned graph has a volume of 6 after *B* is moved to the blue partition . . 27
4.8 Partitioned graph has a volume of 8 after *B* is moved to the green partition . 27
4.9 Partitioned graph with a total communication volume of 8.. . . . . 27
4.10 Partitioned graph has a volume of 5 after the move. . . . . 28
4.11 Partitioned graph has a volume of 6 after the move. . . . . 28

7.1 Standard deviation normalized by the average volume for runs with $k = 2$ on the Heart dataset. VOL (blue) shows nonzero variation across graphs; NVOL (orange) is 0 on every graph, meaning its two partitions have identical outgoing volume.. . . . . 46
7.2 Comparison of the outgoing communication volumes of the partitions of heart04 $k = 4$ for VOL (blue) and NVOL (orange). . . . . 48
7.3 Percentage difference in $W^{\max}$ between NVOL and VOL for italy_osm graph across different $k$. Each value is the result of $(W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100$. 51
7.4 Comparison of the average normalized standard deviation on coPapersDBLP across different $k$ for VOL (blue) and NVOL (orange). . . . . 52
7.5 Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 8$ for VOL (blue) and NVOL (orange).. . . . . 52

7.6 Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 128$ for VOL (blue) and NVOL (orange). . . . . 53
7.7 Comparison of the average normalized standard deviation and total communication volume on kmer_V2a across different $k$ for VOL (blue) and NVOL (orange). . . . . 54
7.8 Comparison of the average normalized standard deviation and total communication volume on kmer_U1a across different $k$ for VOL (blue) and NVOL (orange). . . . . 55
7.9 Comparison of the outgoing communication volumes of the partitions of kmer_U1a $k = 128$ for VOL (blue) and NVOL (orange).. . . . . 56
7.10 Percentage difference in $W^{\max}$ between NVOL and VOL for our selection of geometric graphs across different $k$ $((W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100)$. . . 58
7.11 Estimated runtimes per partition on heart06 $k = 32$ for VOL and NVOL. The value after NVOL represents the percentage difference between the highest value for VOL and NVOL. . . . . 62
7.12 Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 32$ for VOL (blue) and NVOL (orange) with 4% imbalance. 63
7.13 Comparison of the incoming communication volumes of the partitions of coPapersDBLP $k = 32$ for VOL (blue) and NVOL (orange) with 4% imbalance. 64
7.14 Estimated runtimes per partition on italy_osm $k = 32$ for VOL (blue) and NVOL 4% (orange). The value after NVOL is the percentage difference between the highest runtime for VOL and for NVOL. . . . . 65
7.15 Estimated runtimes per partition on italy_osm $k = 32$ for VOL (blue) and NVOL 4% (orange) when ignoring the latency. The value after NVOL is the percentage difference between the highest runtime for VOL and for NVOL. . . . . 65

# List of Tables


4.1 Notation used in Chapter 4. . . . . 19

7.1 Heart dataset [34]. . . . . 44
7.2 Selection of matrices from SuiteSparse Matrix Collection [11] . . . . . 44
7.3 Percentage difference in $W^{\max}$ between NVOL and VOL for the Heart dataset across different $k$. Each value is the result of $(W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100$. Cells marked with N/A were not run. . . . . 45
7.4 Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL (Heart dataset). . . . . 45
7.5 Comparison of the standard deviation of the outgoing volumes and total volume on heart04 graph for VOL and NVOL . . . . . 47
7.6 Percentage difference in $W^{\max}$ between NVOL and VOL for the SuiteSparse Matrix Collection selection across different $k$. Each value is the result of $(W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100$. Cells marked with N/R and N/A were not run. . . . . 48
7.7 Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL (SuiteSparse Matrix Collection). . . . . 49
7.8 Comparison of the standard deviation of the outgoing volumes and total volume of VOL and NVOL for $k = 2$ runs where NVOL is worse in Table 7.6.. . . . 49
7.9 Comparison of the percentage difference of total volume and standard deviation on hugetric-00000 for VOL and NVOL. . . . . 57
7.10 Percentage difference in $W^{\max}$ between NVOL 4% and VOL for the Heart dataset across different $k$. Each value is the result of $((W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/A were not run. . . . . 59
7.11 Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL 4% (Heart dataset). . . . . 59
7.12 Percentage difference in $W^{\max}$ between NVOL 4% and VOL for the SuiteSparse Matrix Collection selection across different $k$. Each value is the result of $((W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/R and N/A were not run. . . . . 60
7.13 Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL 4% (SuiteSparse Matrix Collection). . . . . 60
7.14 Percentage difference for the slowest runtime of NVOL and NVOL 4% compared to VOL (e.g. $\Delta R_{\text{NVOL}} = (R_{\text{VOL}} - R_{\text{NVOL}})/R_{\text{VOL}}$). . . . . 62
7.15 Communication statistics for VOL and NVOL 4% on italy_osm with $k = 32$. . 64

# List of Algorithms


1 Total gain calculation . . . . . 29
2 Partition specific gain calculation . . . . . 30
3 Total communication volume refinement . . . . . 32
4 NVOL refinement . . . . . 35

# Acronyms

| | |
|---|---|
| **VOL** | Total Volume objective function |
| **NVOL** | New Volume partition-aware objective function |
| **GP** | Graph Partitioning |
| **GPP** | Graph Partitioning Problem |
| **BFS** | Breadth-First Search |
| **SGP** | Streaming Graph Partitioning |
| **GGP** | Graph Growing Partitioning |
| **KL** | Kernighan-Lin |
| **GR** | Greedy Refinement |
| $W^{\max}$ | Maximum Outgoing Partition Volume |

# Chapter 1

# Introduction

The demand for computational resources in modern scientific and engineering applications keeps increasing, making parallel computing an essential area of development. While processor performance continues to improve, the growth in single-core performance has slowed considerably since the end of Dennard scaling [22], making it impractical to tackle the largest problems on a single processor alone. Instead, it has become necessary to distribute these workloads across the many processors of a parallel computing system.

In order to compute large communicating tasks in parallel computing it is necessary to distribute the tasks on different machines. To facilitate this, many models have been proposed that represent communicating tasks as graphs or hypergraphs [5, 6, 8, 19, 47]. The vertices correspond to units of computation and the edges encode data dependencies between them [19]. Efficiently distributing the computation then becomes a question of how to partition this graph among processors: balancing the workload evenly while minimizing the communication between them. This is also known as the *graph partitioning problem*. Although graph partitioning is known to be NP-complete, a variety of algorithms have been proposed that produce good quality solutions in practice by using heuristics [20, 25, 27, 28]. Beyond parallel computing, graph partitioning plays a critical role in numerous other application areas, including task scheduling, neural network simulations, operations research and more.

More than 20 years ago the scientific community already argued that the future of software development lies in concurrency, e.g [44]. As computational demands continue to grow, our programs and requirements consistently outpace hardware improvements. Sutter highlights the need for better parallel programming tools, given the inherent difficulty of developing parallel solutions. A decade later, on July 29, 2015, President Obama issued Executive Order 13702, establishing the National Strategic Computing Initiative (NSCI) to ensure continued United States leadership in high-performance computing [13], underscoring the importance of this field and the need for sustained research. Graph partitioning algorithms are central to making this possible.

Most modern graph partitioning algorithms minimize for *edge- cut* (see Section 2.1.1), seeking to minimize the number of inter-partition edges. The appeal of this metric lies in its simplicity: it is easy to compute, and for the structured meshes that dominate scientific computing, it correlates reasonably well with actual communication costs [19]. However, edge-cut has several well known shortcomings as a representation for communication [18, 19]. First, it over counts *communication volume* (see Section 2.1.1): a vertex with multiple edges crossing into the same partition needs to be communicated only once, yet each edge is counted separately when using edge-cut. Second, the

metric ignores data communication latency, the startup cost when sharing data between partitions, which can dominate communication cost when the size of the communicated data is small. Third, and most relevant to this work, minimizing the *total* edge-cut does not account for how communication is distributed across partitions. In a parallel application, performance is generally limited by the slowest processor [19], meaning that even if the total edge-cut is good, some processors might bear a greater load than others, creating bottlenecks.

Karypis and Moulitsas highlighted the need for partitioning algorithms that account for heterogeneous architectures and proposed using *communication volume* as an optimization objective instead of edge-cut [35]. This communication volume objective was subsequently added into the METIS software package. However, like the edge-cut objective, METIS focuses on the total communication volume without considering how volume is distributed among partitions. Regardless of the architecture, focusing only on the total volume of a partitioned graph may lead to bottlenecks in some partitions.

Efficiently distributing computation across processors is central to high-performance computing, and graph partitioning is the standard technique for achieving this. In *Graph Partitioning and Parallel Solvers: Has the Emperor No Clothes?* [18], Hendrickson concludes that one of the key needs is to develop a more accurate metric for communication cost. Researchers have proposed alternative minimization objectives that target per-processor communication metrics rather than global totals. An example is UMPa [8], which minimizes the maximum data sent by any single processor in a distributed memory setting.

## 1.1 Thesis Goal

As Hendrickson noted, minimization objectives that target communication costs are harder to optimize than the standard edge-cut, which partly explains their limited adoption. Despite these difficulties, in this thesis, we present a partition aware communication volume objective called *NVOL*. NVOL minimizes the *per-partition* maximum volume, aiming to reduce uneven communication loads. Using the METIS software as our baseline multilevel $k$-way partitioner, we implement this new partition-aware objective and address two questions: Can a partition-aware objective produce better balanced communication loads than a global one? Can the performance of a parallel application be improved in practice by using a partition-aware objective?

NVOL is implemented in the refinement stage of the multilevel $k$-way algorithm. After an initial partition is performed, the partitions are gradually refined by moving vertices between partitions. NVOL gathers information about how every partition's communication volume is affected by a vertex move, allowing it to correctly evaluate how much partition $a$ will change if a vertex $v$ is moved to another partition $b$. Using this information, NVOL prioritizes moves that improve the per-partition maximum. No move that increases the maximum volume held by a single partition is accepted. This means that in some cases NVOL might even accept moves that increase the total communication volume, as long as they decrease the per-partition maximum. The goal is to provide more balanced partitions, avoiding overloading some for the sake of a better total communication volume.

## 1.2 Thesis Structure

**Chapter 2:** *Background*

Chapter 2 covers the fundamentals of graph partitioning. It surveys some of the algorithms in common use today, along with the main application areas in which they are applied.

**Chapter 3:** *Multilevel $k$-Way Graph Partitioning*

Chapter 3 presents the multilevel $k$-way graph partitioning algorithm on which NVOL is built. This approach offers a good balance between partition quality and runtime, making it a popular choice for large graphs. The chapter focuses on the standard formulation, which uses edge-cut as its objective; the following chapters then extend the algorithm to support volume-based objectives such as NVOL.

**Chapter 4:** *Communication Volume Calculation*

Chapter 4 explains how gain is evaluated for volume-based objectives. It covers the calculation of both total communication volume and per-partition volume, beginning with formal definitions of the gain calculations and followed by worked examples. The final sections describe how these calculations are integrated into the multilevel $k$-way algorithm.

**Chapter 5:** *Refinement Using Communication Volume*

Chapter 5 covers the refinement stage and explains how it is implemented for volume-based objectives. Building on Chapter 4, it shows how partitions can be refined using communication volume objectives. The total communication volume objective and NVOL are each treated in their own section, and both conclude with a simplified algorithm representing the implementation.

**Chapter 6:** *METIS and Supporting Tooling*

Chapter 6 presents the tools used in this thesis to run and analyze the multilevel $k$-way partitioning algorithm with NVOL. It introduces METIS, the partitioning framework that NVOL extends, and describes its input and output formats. It also presents a small collection of Python scripts developed alongside this work for preprocessing input graphs and analyzing the per-partition communication volumes produced by the experiments.

**Chapter 7:** *Results*

Chapter 7 presents the results of running NVOL across a variety of graphs and partition counts. It compares the partitions across multiple metrics and models the runtime of executing the partitions produced by NVOL and METIS.

**Chapter 8:** *Future Work*

Chapter 8 presents directions for future work on this project. It discusses some of the current shortcomings and how they might be addressed, and introduces new ideas that would build naturally on the NVOL implementation. The goal of the latter is to extend the move-selection logic in the refinement stage in order to produce better partitions.

**Chapter 9:** *Conclusions*

Chapter 9 concludes the thesis and evaluates the contributions of this project. It revisits the initial goals set out in Chapter 1 and discusses how well they have been fulfilled.

# Chapter 2

# Background

This chapter gives a description of what the *graph partitioning problem* (GPP) is, as well as what objective functions are used to evaluate partitions. It also discusses some of the common practices in the field with some examples of concrete application areas for *graph partitioning* (GP).

## 2.1 Graph Partitioning Problem

The GPP refers to a group of problems that deal with distributing the vertices of a graph into groups, clusters or communities. The goal is to group together vertices that are similar or that share many edges between them to minimize the number of edges between groups. The edges in some graphs are directed, meaning that they contain information about the communication flow between vertices. However, directed edges, or more generally directed graphs, are outside the scope of this thesis, and we consider only undirected ones.

Consider an undirected graph $G = (V, E)$ with $|V| = n$ vertices and $|E| = m$ edges. The goal is to partition $V$ into $k$ disjoint subsets $V_0, V_1, \ldots, V_k$ such that $V_i \cap V_j = \emptyset$ for $i \neq j$ and $\bigcup_i V_i = V$ [27]. The weight of the graph is $W = \sum_{v \in V} w(v)$, where $w(v)$ is the weight of vertex $v$ (vertices can also have multiple weights, but in this thesis we only consider graphs with a single vertex weight). Note that a graph with unit weights will have a weight equal to the number of vertices $W = n$. A balance constraint requires that all subsets have approximately equal weight. The partition weight equals the weight of each vertex it contains, $w(V_i) = \sum_{v \in V_i} w(v)$. Using an imbalance parameter $\epsilon$, this constraint requires that each subset satisfies

$$w(V_i) \leq \left\lceil \frac{W}{k} \right\rceil \cdot (1 + \epsilon), \quad i = 0, 1, \ldots, k.$$

The parameter $\epsilon$ represents the fractional excess weight a subset may have beyond the perfectly balanced weight $\lceil W/k \rceil$. When $\epsilon = 0$, the partition must be perfectly balanced [4]. The goal of GPP is to minimize a given objective function (see Section 2.1.1) while satisfying the balance constraint. GPP is NP-complete, meaning that for large graphs a solution can only be achieved through heuristics or approximation algorithms [17]. Unlike heuristics, which seek good solutions empirically without formal guarantees, approximation algorithms are required to come with a mathematically proven bound on how far their output can deviate from the optimum [48]. Despite their theoretical importance, approximation algorithms either underperform compared

to state-of-the-art GP solvers or are too slow for large graphs [4], making heuristics the most common choice in practice.

Another common representation of computational tasks in GP is *hypergraphs*. A hypergraph is a generalization of a graph where an edge, also called a hyperedge, can connect any number of vertices [3]. Similarly, GP divides the vertices into partitions while trying to maintain roughly equal size for each. However, given this more complex structure, utilizing hypergraphs requires more complex algorithms both in terms of implementation and running time. For the scope of this thesis, we will focus on standard GP; however, most of the techniques presented in this section (see Section 2.2) have been or can be applied to a solution based on hypergraphs as well.

### 2.1.1 Objective Functions

The most common objective in GPP is to minimize the total edge-cut [4]. An edge $\{u, v\} \in E$ is said to be *cut* if its endpoints belong to different partitions, i.e., $u \in V_i$ and $v \in V_j$ with $i \neq j$. The total edge-cut is then the sum of all such cut edges across partitions. This also applies for graphs with edge weights, where the total edge-cut is then the sum of the edge weights of cut edges.

Despite being a less precise measure for actual communication cost, edge-cut has been widely adopted as the standard objective. Buluç et al. [4] attribute this to two factors: cut optimization appears to be easier in practice, and for graphs with high structural locality, the cut tends to perform similarly to other objectives.

Another common objective is communication volume, which provides a more accurate measure of the data exchange between partitions of a communicating task. For a partition $V_i$, the outgoing communication volume is defined as $\text{comm}(V_i) = \sum_{v \in V_i} D(v)w(v)$, where $D(v)$ denotes the number of distinct partitions, other than the one containing $v$, in which $v$ has at least one edge to another vertex. The communication volume is then multiplied with the vertex weight, $w(v)$, as that represents the outgoing communication volume.

### 2.1.2 Shortcomings of the Objective Functions

Even though edge-cut is accepted as the standard, it comes with several shortcomings. Most notably, it does not distinguish between multiple edges connecting to the same neighboring partition. Consider Figure 2.1, where each color represents a partition. Vertex $A$ has two edges, both leading to the blue partition. Calculating the edge-cut, $A$ contributes 2 to the total edge-cut. The communication volume, however, is only 1, since both edges connect to the same partition. In general, edge-cut can overestimate the actual communication burden, particularly for vertices with many neighbors in a single remote partition.

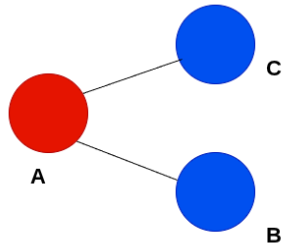


Figure 2.1: Vertex $A$ has an outgoing communication volume of 1 but an edge-cut of 2. Each color represents a partition.

The performance of a parallel application is generally limited by its slowest processor. This means that even if the computational work is balanced, the communication effort may not be [4]. In such cases, it is more meaningful to minimize the maximum communication load incurred on a single processor rather than the total load across all processors. Neither the standard edge-cut metric nor the total communication volume captures this type of objective.

Furthermore, on many architectures the time to send data, or messages, depends not on geographic distance but on the number of network switches the data must traverse. Although modern machines employ cut-through or wormhole routing to move individual messages quickly between distant processors, the communication network handles many messages simultaneously, and a message between distant processors occupies links that are unavailable to other traffic. Minimizing edge-cut does not account for this locality of communication, and a partition that places frequently communicating processors far apart in the network topology can lead to significant message contention [4, 19].

Finally, the time to send a message on a parallel computer depends on both the latency (startup cost) and the message size. GP approaches typically aim to minimize the total communication volume but not the total number of messages sent. Depending on the machine architecture and problem size, message latencies can dominate message volume, making the number of distinct messages a relevant cost factor as well [4, 19]. A few big messages might be more efficient than many small ones.

Regardless of the objective function chosen, GP remains a fundamental tool across a wide range of domains. The following sections discuss some of the most used GP algorithms and some of their application areas.

## 2.2 Graph Partitioning Algorithms

In this section we will present some of the most popular GP algorithms used today. The goal is to show some of the differences in the approaches and to highlight their typical use cases. This will help the reader to understand why the multilevel partitioning approach is deemed the best option for large scale graphs. These are, however, only *some* of the existing GP algorithms, a more complete list can be found in [4, 7].

### 2.2.1 Recursive Bisection

A popular partitioning method is recursive bisection (RB). Instead of partitioning a graph directly into $k$ subsets, RB first bisects the graph into two roughly equal parts using algorithms such as [1, 14, 30], and then recursively bisects each part until $k$ partitions have been formed. When $k$ is not a power of two, modified schemes are used [9, 20,

26, 41]. Although simple and widely used, RB has been shown to produce suboptimal $k$-way partitions [41], since each bisection is made independently without knowledge of the cuts that will follow at deeper levels of the recursion.

### 2.2.2 Spectral Bisection

Spectral Bisection was one of the first methods used to split a graph into two partitions and is still used today [4]. Its approach is to use the eigenvector, also known as the *Fiedler vector*, of the second smallest eigenvalue of the Laplacian matrix to deduce global information of the connectivity of the graph [15]. In order to form the two partitions, the median value $\overline{m}$ of the Fiedler vector is calculated, and every vertex with a value smaller or equal to $\overline{m}$ is assigned to one partition, and the rest to the other one.

The main computational cost of the spectral method lies in the calculation of the Fiedler vector. To optimize this, Barnard and Simon [1] have used a multilevel method to produce an approximation of the Fiedler vector. The spectral method has also been extended to produce a partition for more than only two partitions and to support edge and vertex weights [20].

### 2.2.3 Graph Growing Partitioning

Another bisection approach is called graph growing partitioning (GGP) [26]. GGP is mainly based on breadth-first search (BFS). The core idea is to start from a random vertex and to use BFS to construct a partition. BFS is run until half the vertices are assigned to one partition, the remaining vertices are then assigned to the other one. The choice of starting vertex significantly affects the result, meaning that this algorithm tends to require multiple restarts to obtain a good solution. The GGP approach has also been extended to produce more than two partitions, called *Bubble framework* [12]. In this version $k$ starting vertices are chosen, evenly distributed in the graph. Starting from these vertices, $k$ BFS traversals are performed. Throughout, *Bubble* uses local search algorithms to balance the partitions in the graph. At the end of an iteration, a new starting vertex is chosen for all partitions. The new starting vertex is the vertex with the minimum total distance to all other vertices in the same partition. The algorithm is repeated until no new changes occur in the choice of starting vertices, or no improvement for the partitions is found. The major drawback of this approach is its high computational cost.

### 2.2.4 Streaming Graph Partitioning

Streaming data models, where input is processed on the fly rather than all at once, have gained popularity due to their small memory requirements. In the field of GP, the input graph is also passed as a stream, requiring much less space than the graph size [4]. Streaming graph partitioning (SGP) is also very fast, even faster than multilevel algorithms (see Section 2.2.5). However, these algorithms produce lower quality partitions compared to multilevel approaches. They are nonetheless useful in applications where speed is important, and an initial partition can be obtained from a stronger GP algorithm. For more details about SGP, we refer the reader to [38, 43, 46].

### 2.2.5 Multilevel Graph Partitioning

Multilevel GP is considered to be the most successful heuristic for partitioning large graphs [4, 26, 28]. Because of this, multilevel GP was chosen as the base on which to

build our new objective function NVOL. In short, the idea is that a graph is coarsened to a smaller graph. Once the graph is at its smallest, an initial partition is performed, and then it is iteratively uncoarsened to the original size. During the uncoarsening, heuristics are applied to improve the partitions. The intuition behind multilevel GP is that it is easy to run intensive partitioning algorithms on the small graph without affecting the total runtime. Additionally, starting from an already good solution, the refinement stage is not costly to perform and can quickly improve the partitions. Chapter 3 will explain in detail the algorithm, providing the necessary background for understanding how NVOL is integrated into the framework.

## 2.3 Applications of Graph Partitioning

Graph partitioning arises in a broad variety of domains. This section highlights several of the most prominent application areas, though the list is by no means exhaustive.

### 2.3.1 Parallel Processing

Perhaps the most widespread application of GP is distributing computational work across the processors of a parallel machine [31]. GP is the standard approach in scientific computing applications such as sparse direct and iterative solvers. When the problem domain remains fixed throughout the computation, the partitioning can be performed once at the outset; this is known as static partitioning [4]. Other common applications in parallel processing include eigenvalue computations [2], breadth-first search [3], and algorithms for PageRank and connected components [40].

### 2.3.2 Complex Networks

GP is also relatively popular in the field of complex networks [4]. Complex networks, in this context, refer to weighted graphs with nontrivial structure arising from real world or modelling processes [37]. The objectives applied in this field are usually domain specific, making it hard to develop optimization algorithms.

Several application domains illustrate this breadth. In power grids, disturbances and cascading failures may lead to blackouts. GP can be used to isolate self-sufficient islands to prevent propagation of an outage [32]. Cut based objectives are sometimes extended to enhance the robustness of the system [33].

Geographically embedded networks have experienced a growth in requirements with the advances in location-aware devices (such as GPS). It is now vital that algorithms be able to analyze networks like these extremely fast. In these networks the vertices commonly represent entities in the real world tied to geographic places. The cut-based approach therefore needs to be reinforced with spatial contiguity constraints [4, 33].

Many biological systems can be modeled as graphs [4], protein-protein interaction or gene co-expression graphs, for example. Partitioning serves multiple purposes ranging from data reduction to the detection of biological processes. More about this field can be found in [23].

In social networks, community detection is one of the most popular topics. In contrast to traditional GPP, the number of subgroups or partitions is not known a priori [4]. Apart from this, GP has contributed greatly to the development of community detection algorithms [16, 36].

### 2.3.3 VLSI Physical Design

VLSI circuit design has historically been one of the primary application areas for graph partitioning [4]. The objective is to decompose a circuit into smaller subcircuits, ranging from gate arrays to complete integrated circuits, while minimizing the total interconnection cost, typically measured as the weight of connections crossing partition boundaries. In this formulation, nodes represent cells (i.e., basic logic or functional units such as gates) and edges represent wires. Scalability is essential, as circuits can contain millions of modules and partitioning constitutes a bottleneck in the overall design flow. Beyond the basic cut objective, practical VLSI partitioning must also respect domain specific constraints such as I/O placement, groups of cells that are required to remain in the same block, and upper bounds on the cut size between block pairs. Further details can be found in [10, 24].

### 2.3.4 Neural Networks

Graph neural networks (GNNs) extend deep learning to graph structured data, enabling the processing of irregular domains that traditional neural network architectures cannot readily handle. However, as graph sizes grow, training GNNs becomes increasingly expensive due to rising communication overhead, particularly in distributed or multi-device settings. Graph partitioning offers a natural approach to this scalability challenge by distributing subgraphs across devices while minimizing inter-device communication.

A recent example is CaPGNN [42], a parallel full batch GNN training framework designed for multi-GPU servers. CaPGNN combines resource aware graph partitioning with an adaptive caching mechanism that exploits both CPU and GPU memory to reduce redundant data transfers. By taking into account the heterogeneous computational and communication capacities of individual GPUs, the framework achieves substantial reductions in training time with minimal impact on model accuracy. While this is only one instance of a broader trend, it illustrates how classical graph partitioning techniques are finding renewed relevance in the context of modern machine learning workloads.

Another recent example is ParDNN [39], an automatic, generic, and non-intrusive partitioning strategy for Deep Neural Networks represented as graphs. Like CaPGNN, this strategy has the goal of minimizing the training time of the model without affecting accuracy. Through their experiments, Qararyah et al. show significant improvements compared to related work in efficiency, superlinear scaling of batch size and training throughput.

# Chapter 3

# Multilevel K-Way Graph Partitioning

This chapter presents the Multilevel $K$-way Graph Partitioning algorithm which is one of the most widespread GP algorithms. Each section will explain in detail the phases of the multilevel approach with special focus on the techniques used in the partitioner used in this thesis.

## 3.1 Preliminaries

The overall structure of a multilevel $k$-way graph partitioning algorithm is fairly straightforward. The process begins by coarsening the input graph until it is reduced to a sufficiently small size. At this coarsest level, an initial $k$-way partitioning is computed using a graph growing algorithm (see Section 2.2.3) [26]. The graph is then uncoarsened back to its original size in successive steps. At each level of uncoarsening, the partitioning is refined to improve quality by exploiting the increased degrees of freedom available at the finer level. During this phase, the partitions are improved using an objective function. NVOL is implemented in this phase, which is discussed in Chapter 5. Karypis and Kumar demonstrate in [27] that this multilevel approach yields high-quality partitions, often comparable to or better than those produced by recursive bisection [26].

In Figure 3.1, this process is illustrated, showing how an initial graph $G_0$ is coarsened until it reaches $G_4$, which is smaller in size. At this level, an initial partition is performed on the coarsest graph. After that, the graph is step by step projected back to its original size, refining the partition at each level in order to minimize the chosen objective function [29].

The multilevel partitioning problem uses the same definitions as in Section 2.1. A $k$-way partition is represented through a vector $P$. The vector has a length of $n$, where $n$ is the number of vertices in the graph. Each element of $P$ is an integer between 1 and $k$, indicating the partition the vertex belongs to. Specifically, a vertex $v \in V$ assigned to partition $a$ will have $P[v] = a$. The next sections will now explain how each phase works and how the partitions are formed.

## 3.2 Coarsening Phase

During the coarsening phase, a sequence of increasingly smaller graphs is constructed, maintaining a similar structure to the original graph with fewer degrees of freedom. The most common approach is to aggregate groups of vertices into a single coarse vertex through matching [4, 26]. Formally, the idea is to contract pairs of matched vertices, where we define a matching as a set of edges no two of which share a vertex [4, 26].

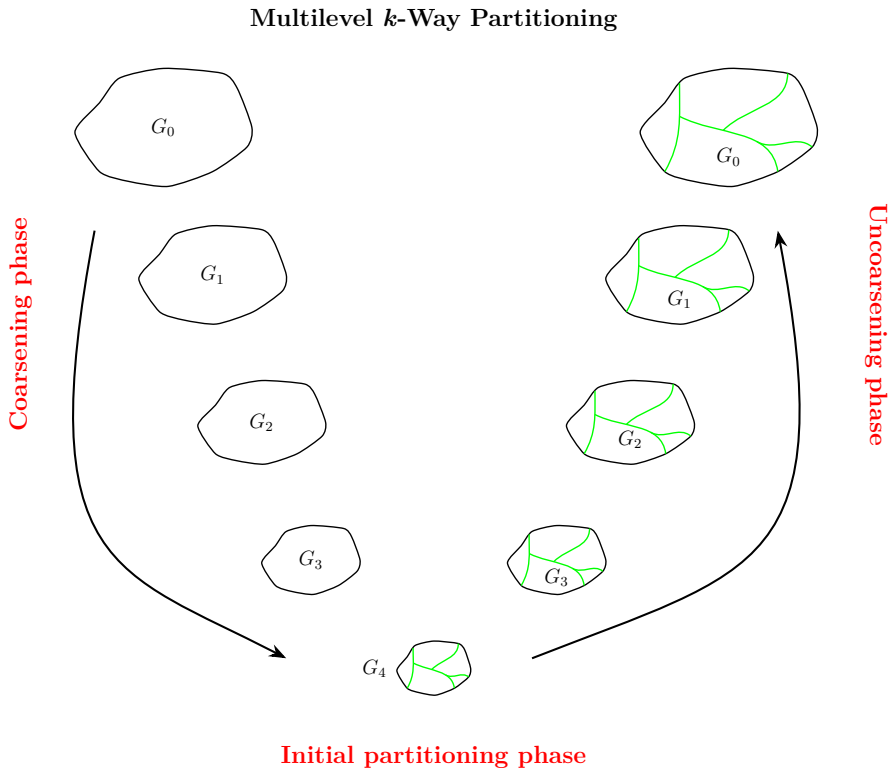


Figure 3.1: The phases of the multilevel $k$-way partitioning algorithm. During the coarsening phase, the vertices are contracted together and the graph size is successively decreased; during the initial partitioning phase, a $k$-way partition is performed on the smallest graph; and during the uncoarsening phase, the partitions are successively refined and projected back to a larger graph.

Since the goal of coarsening is to reduce the graph size, a matching is considered good if it is composed of many matched edges. To achieve this, *maximal matchings* are used. A matching is maximal if any edge in the graph that is not contained in the matching has at least one of its endpoints matched [26]. The following subsection describes in detail the implementation of Sorted Heavy Edge Matching (SHEM), which is the matching algorithm employed by the partitioner used in this thesis. For further reading on other coarsening algorithms, we refer the reader to [4, 26].

Once a matching is performed the coarse vertex has a weight equal to the sum of the weights of all the vertices it contains. Additionally, the coarse vertex receives all the edges of the vertices it contains. Vertices that are not matched are simply copied over to the smaller graph. This process is repeated until we reach a graph $G_m$ with a small enough number of vertices [25].

For a given graph $G_i = (V_i, E_i)$, a subset of vertices $V_i^v \subseteq V_i$ is merged into a coarse vertex $v \in V_{i+1}$ for the next coarser graph $G_{i+1}$. The coarse vertex $v$ has weight $w(v) = \sum_{u \in V_i^v} w(u)$. This ensures that the total vertex weight is preserved across coarsening levels. To preserve connectivity, the new vertex $v$ receives the union of the edges of the vertices in $V_i^v$. For any neighboring vertex $u$, the edge weight is $w(\{v, u\}) = \sum_{z \in V_i^v} w(\{z, u\})$. This aggregation strategy preserves structural and weight-related properties of the original graph, thereby enabling efficient and balanced processing at coarser levels [27].

### 3.2.1 Sorted Heavy Edge Matching

SHEM begins by generating a random permutation of the graph's vertices, which is then sorted based on vertex degrees. Each vertex gets assigned a bucket based on the square of its degree, sorted from lowest to highest. This ordering ensures that low-degree vertices, which have fewer opportunities for matching, are prioritized and thus more likely to be successfully matched. Delaying them would increase the chance of remaining unmatched. Once the vertices are sorted, they are traversed in order. For each unmatched vertex $u$, the algorithm selects the unmatched neighboring vertex $v$ connected to $u$ via the heaviest incident edge. The edge $\{u, v\}$ is then added to the maximal matching $M_i$, and both $u$ and $v$ are marked as matched. If no suitable neighbor exists, the vertex $u$ remains unmatched. This process continues until all vertices have been processed.

The greedy nature of SHEM tends to preserve strong edge connections in the coarser graph, contributing to a significant reduction in total edge weight and, consequently, in the number of edge cuts [26, 28]. As shown in [25] by Karypis and Kumar, since the coarser graph has lower edge weight, it also has a lower edge cut.

## 3.3 Initial Partitioning Phase

Once the graph has been coarsened to the smallest level, the initial partitioning is performed. The goal of this stage is to produce $k$ roughly equal partitions before further refinement. Several partitioning algorithms can be used here, including spectral partitioning [20], graph growing partitioning [26], and multilevel recursive bisection [41], among others. Since the graph is relatively small at this stage, the partitioning is fast. The partitioner we used, METIS, uses a multilevel recursive bisection approach. It is also worth mentioning that METIS uses different bisection algorithms based on the graph type. Using the definition in Section 2.1 we use GGP as the default bisection algorithm.

The a multilevel recursive bisection approach takes the coarse graph and bisects it using GGP. The resulting two subgraphs are then each bisected again. This process is repeated recursively until the desired number of partitions is reached. When the number of partitions is not a power of 2, the algorithm will keep bisecting until one graph has 3 partition remaining. At this level only one of the produced subgraphs will be further bisected. Consider Figure 3.2 where graph $G$ is to be partitioned into 7 groups. At the first level, $G_0$ has 4 partitions remaining while $G_1$ has 3. Since $G_1$ has only 3 partitions remaining, only $G_4$ is bisected further. The left side ($G_0$), on the other hand, continues bisecting until all partitions have been produced.

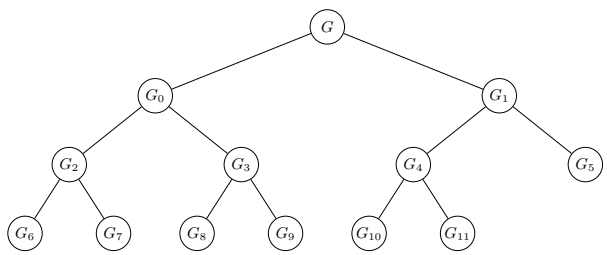


Figure 3.2: The process of bisecting a graph into 7 partitions.

During each bisecting step, GGP is used. As it was described in Section 2.2.3, this bisection approach bases itself on BFS. Starting from a random seed it tries to group together vertices connected to each other. At every step it checks that both partitions maintain a weight threshold. Once a partition is computed the graph is further refined

using the refinement algorithm proposed by Fiduccia and Mattheyses (FM) in [14]. The FM refinement algorithm goes through the boundary vertices of the graph, vertices that share at least one edge with a vertex in the other partition. For each step the vertex move that yields the greatest reduction in edge cut (see Section 2.1.1) is selected. In order to avoid a subpar solution, FM allows a limited number of moves that worsen the edge cut. This process is repeated over the boundary vertices several times, each time keeping the partition with the lowest edge cut.

Regardless of the non-deterministic nature of the GGP algorithm, METIS will produce the same result for identical re-reuns. METIS uses a fixed default seed, so the "random" choice is the same across re-runs. In other words, if the seed is not manually passed, the initial partition will always be the same when running a graph with same parameters multiple times.

It is worth noting that also for our volume based objective we have made use of this approach based on edge cut for the initial partition. There are a few reasons for this. Firstly, the initial partition is going to be heavily refined later during the uncoarsening phase, making the quality of the initial partition not as important. Additionally algorithms like FM are fast and work well with edge cut, optimizing for another objective would make this phase significantly slower. All of these reasons make it reasonable to use edge cut as an objective for this first partitioning.

## 3.4 Uncoarsening Phase

The final step is the uncoarsening and refinement phase, during which the graph is gradually projected back to its original size. The coarsest graph $G_m$ is successively expanded through finer representations $G_{m-1}, G_{m-2}, \ldots, G_1$. At each projection from $G_{i+1}$ to $G_i$, every vertex contained within a coarse vertex $v$ inherits the partition assignment that $v$ held in $G_{i+1}$. Once all coarse vertices at a given level have been expanded, the resulting partition is improved using local refinement heuristics.

Depending on the objective function chosen, the refinement heuristics will be different. In the rest of this section, we present the standard approach used by the partitioner we use, METIS, based on edge cut. The edge cut based refinement is simpler to understand, and will help the reader to get a better idea of how refinement works. In Chapter 4 and Chapter 5 we will discuss the total communication volume objective and present our expanded volume objective NVOL.

METIS employs a variant of the Kernighan-Lin (KL) algorithm [30] to refine the graph. The KL algorithm iteratively swaps vertices among partitions of a bisection to minimize edge cut, continuing until no further improvement is possible and a local minimum is reached [27]. Refining a $k$-way partition is significantly more complex since each vertex has $k - 1$ possible moves as opposed to just one in a bisection[27]. This substantially enlarges the optimization space, forcing the algorithm to calculate the gain in each possible combination, thus increasing the computational complexity. The gain represents the change in total edge cut resulting from relocating a vertex to a new partition.

### 3.4.1 Projection and preparation before refinement

At every level of uncoarsening the graph is projected to a finer level. All the coarse vertices are split open, assigning each finer vertex to the same partition the coarse vertex had. Once the graph is projected up a level it undergoes a preparation phase before

actual refinement. The new finer graph contains new vertices and edges, requiring a full recomputation of the underlying structures describing the graph. During this phase the whole graph is therefore traversed in order to get accurate information about the new connections in the graph. Additionally, the boundary set is recomputed, making sure to account for new boundary vertices.

The boundary set defines the list of vertices visited during refinement. So, to further increase efficiency METIS filters this set by more than wether a vertex is on the boundary or not. As the next section will show, a move is accepted only if it leads to positive or zero gain. Therefore, METIS chooses to ignore boundary vertices with negative gain.

In order to express gain for edge cut we need to introduce two more definitions. Any vertex has an internal and an external degree, which define the sum of the edge weights the vertex has toward different partitions. The internal degree is the sum of the weights of edges from a vertex $v$ to other vertices in the same partition as $v$. We denote the internal degree of a vertex $v$ with $ID[v]$. The external degree is the sum of the weights of edges from $v$ to vertices in partitions other than its own. We denote the external degree of $v$ as $ED[v]$. To simplify calculations during refinement we also specify the external degree towards a specific partition $a$ as $ED_a[v]$.

A vertex $v$ is then added to the boundary set if:

$$ED[v] - ID[v] \geq 0.$$

The specific external degree of a vertex will always be smaller or equal to the total external degree. Using Equation 3.1 we see that the gain will be positive only if the specific external degree is greater than the internal degree. Consequently, if the total external degree is already less than the internal degree no move will possibly lead to positive gain.

To optimize refinement even more, the boundary set is ordered by the potential gain a vertex has. Since the actual gain needs to be calculated for every possible move a vertex has, METIS makes an estimate for this potential gain. For a vertex $v$ this potential gain is expressed as:

$$\frac{ED[v]}{\sqrt{|N_v|}} - ID[v]$$

where $|N_v|$ is the number of unique partitions $v$ is connected to. This gives a normalized result of what the probable gain is. Using this ordering increases the chance of performing moves with high gain early.

### 3.4.2 Greedy Refinement

Karypis and Kumar propose an improved version of the $k$-way refinement KL algorithm in [27] referred to as Greedy Refinement (GR). This algorithm achieves high quality partitions while maintaining a computational complexity that is independent of the number of partitions $k$. Traditional KL-based refinement algorithms implement a lookahead strategy, which permits the movement of clusters of vertices across partition boundaries [27]. However, within the context of a multilevel partitioning framework, the benefit of this mechanism diminishes. Since each vertex in a coarser graph $G_i$ represents a cluster of vertices from a finer graph $G_{i-1}$, moving a single vertex in $G_i$ already corresponds to relocating a group of vertices in the original graph. Consequently, the lookahead mechanism becomes redundant. GR eliminates the lookahead step, thereby simplifying the refinement process and infrastructure presented in [21]. This

simplification contributes to both the reduced computational complexity and the practical efficiency of the GR algorithm.

The GR algorithm proceeds through multiple iterations, where each iteration involves visiting all boundary vertices. For each vertex, the algorithm determines whether relocating it to a neighboring partition would improve the total edge cut. GR might in some cases also choose moves that do not change the edge cut, but that lead to an improved balance in the graph. Any move must satisfy the balancing conditions [27]. The updated partitioning serves as the input for the next iteration.

For a graph $G_i = (V_i, E_i)$ and its partitioning vector $P_i$, each vertex $v \in V_i$ has a set of neighboring partitions $N_v$. That is, the union of the partitions that the vertices adjacent to $v$ belong to. We denote the set of adjacent vertices to $v$ as $V_v^{\text{adj}}$, we can now define: $N_v = \bigcup_{u \in V_v^{\text{adj}}} P_i[u]$ [27]. If all neighbors of $v$ are in the same partition as $v$, then $v$ is considered an *interior vertex*, and $N_v = \emptyset$. Conversely, if $v$ has neighbors in different partitions, then $N_v \neq \emptyset$, and the cardinality of $N_v$ can be as large as $V_v^{\text{adj}}$ (in the case where each neighbor belongs to a distinct partition). During refinement, a vertex $v$ can move to any of the partitions present in $N_v$.

For all boundary vertices we consider how a move to a neighbor partition ($N_v$) affects the total edge cut of the graph. For a boundary vertex $v$, moving to a neighbor partition $a \in N_v$ the gain is defined as:

$$\text{gain} = ED_a[v] - ID[v]. \tag{3.1}$$

GR will iterate over all partitions in $N_v$ looking for any move that yields a positive gain. The move yielding the highest gain is then chosen as destination.

To maintain balance, each move needs to either follow the balance constraints of the graph or to improve the balance of the graph. The weight boundaries are defined as follows:

$$W^{min} = \frac{W}{k(1+\epsilon)}$$

$$W^{max} = \frac{W}{k} \cdot (1+\epsilon)$$

meaning that any partition $V_i$, must be $W^{min} < w(V_i) < W^{max}$ also after a move [27]. A small side note on the notation for $W^{min}$, which is different than the one found in [27], is that we based ours on the source code of the partitioner. Their only difference lies in the new expression using the balance constraint $\epsilon$ instead of a constant like in [27]. Additionally, as a tiebreaker for moves that yield the same gain, GR accepts moves that improve the load balance in the graph. A move improves balance if its destination is a lighter partition, promoting a more equal distribution of vertex weights across partitions [27]. For a vertex $v$ we express it as:

$$\frac{w(\text{destination})}{k} < \frac{w(\text{source}) - w(v)}{k}.$$

After each accepted move the boundary set, neighbor partition sets, and the degrees of the affected vertices are updated. GR will go through the boundary set until all vertices in it have been visited. Note that whenever a move is made the set is affected and vertices may get added or deleted from it. Additionally it's also worth mentioning that moved vertices are not allowed to be moved again during the same refinement stage [27]. Once a full pass is completed the boundary set is computed again and the process repeats itself. By default in our partitioner this process will repeat itself 10 times

with the exception of an early termination if the pass led to no improvement in edge cut or no vertices were moved. Based on their empirical results, Karypis and Kumar observed that GR typically converges within four to eight iterations [27]. This relatively fast convergence contributes to the overall efficiency of the multilevel partitioning approach.

# Chapter 4

# Communication Volume Calculation

This chapter will show how the change in communication volume is calculated. The chapter will begin by explaining how the total communication volume objective works, followed by a definition of how our new partition-aware objective, NVOL, works. This chapter will only focus on how gain is evaluated and implemented in the multilevel framework for communication volume. The next chapter will show how a vertex move is chosen using the results from this calculation.

The total communication volume objective seeks to improve the total communication volume of a partitioned graph. Even though this metric is more representative of actual communication than edge cut, it does not necessarily produce a well balanced result. The net change in total communication volume after a potential move might hide bottlenecks and overloaded partitions. In contrast, NVOL delivers specific information about how the volume of each individual partition is affected by a move. Since NVOL was implemented on top of the METIS framework we will first spend some time formalizing how METIS and other partitioners use total communication volume as an objective in Section 4.2. In Section 4.3 we present the logic behind NVOL, and how we are able to evaluate how partitions are affected by a potential move.

| **Symbol** | **Description** |
|---|---|
| $G$ | The graph |
| $V$ | Set of all the vertices in the graph |
| $E$ | Set of all edges in the graph |
| $V_b$ | Set of all boundary vertices in the graph |
| $V_i^{\text{adj}}$ | Set of all vertices incident on vertex $i$ |
| $N_i$ | Set of neighboring partitions of vertex $i$ |
| $N_{iv}$ | Set of partitions neighboring both vertex $i$ and vertex $v$ |
| $N_{i-v}$ | Set of partitions only neighboring vertex $i$ and not vertex $v$ |
| $w(i)$ | Vertex weight of vertex $i$ |
| $P[i]$ | Partition of vertex $i$ |
| $C_i[n]$ | Number of edges a vertex $i$ has to a partition $n$ |
| $G_i$ | Gain vector of a vertex $i$ |
| $SG_i$ | Gain table of a vertex $i$ |

Table 4.1: Notation used in Chapter 4.

## 4.1 Basic definitions

To facilitate understanding of the implementation (see Section 4.5), we begin by introducing a formal definition of communication volume. Let $G = (V, E)$ be a graph, and let $P$ be the partition vector, where $P[v]$ is the partition a vertex $v$ is assigned to. A vertex $u$ is defined as a neighbor of $v$ if there exists an edge $\{u, v\} \in E$. Any vertex adjacent to at least one vertex belonging to a different partition is said to lie on the boundary. We denote the set of all boundary vertices by $V_b \subseteq V$. Formally, a vertex $v \in V_b$ if there exists at least one neighbor $u \in V$ such that $\{v, u\} \in E$ and $P[v] \neq P[u]$.

For each boundary vertex $v \in V_b$, let $N_v$ denote the set of distinct partitions (excluding $P[v]$) hosting its neighboring vertices. The number of partitions contained in $N_v$, defined as $|N_v|$, is the outgoing communication volume of vertex $v$. Note that the number of edges incident to $v$ is not directly relevant for this measure, only the number of distinct neighboring partitions. In Figure 4.1, vertex $A$ has a communication volume of 2, as it is connected to vertices in two distinct partitions. In contrast, Figure 4.2 illustrates a case where vertex $A$ has two edges, both connecting to vertices within the same partition. Consequently, the communication volume of vertex $A$ is 1, since only a single distinct external partition is involved.

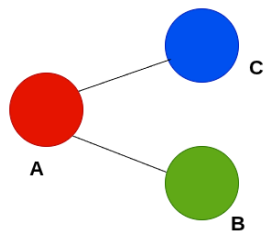


Figure 4.1: Vertex $A$ has an outgoing communication volume of 2. Each color represents a partition.

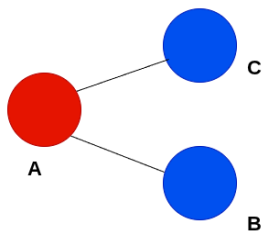


Figure 4.2: Vertex $A$ has an outgoing communication volume of 1. Each color represents a partition.

The total communication volume of the partitioned graph is obtained by summing the outgoing volumes over all boundary vertices:

$$\text{total} \quad \text{volume} = \sum_{v \in V_b} |N_v| \, w(v) \tag{4.1}$$

Here, $w(v)$ denotes the weight associated with vertex $v$, which scales its contribution to the outgoing communication volume. Given the structure of the multilevel approach, regardless of a unit vertex weight graph, the vertices might have weights during uncoarsening (see Section 3.4). In Figure 4.3, all vertices are assumed to have a unit weight. The boundary set is $V_b = \{A, B, C, D, E\}$. Note that vertex $F$ is not part of this set, as it has no edges connecting to vertices in other partitions.

Applying Equation 4.1, the total communication volume is computed by summing the contributions of all vertices in the boundary set. Vertices $A$, $C$, and $D$ each contribute a volume of 2, while vertices $B$ and $E$ each contribute a volume of 1. This results in a total communication volume of 8.

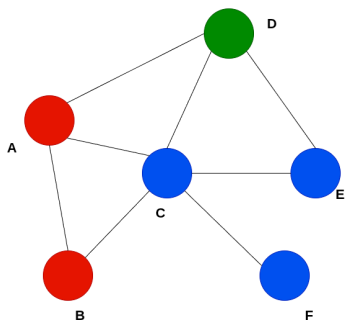


Figure 4.3: The total communication volume is 8.

## 4.2 Total gain calculation

To determine which vertices should be moved, and to which partitions, the gain need to be evaluated for any potential move. The gain represents the change in total communication volume resulting from relocating a vertex to a new partition. A positive gain indicates a reduction in total communication volume, whereas a negative gain corresponds to an increase.

Since communication only arises when an edge connects two vertices belonging to different partitions, a vertex move can only affect the communication volume of its neighboring partitions. For each boundary vertex $i \in V_b$, a partitioner needs to examine all its adjacent vertices $v \in V_i^{\text{adj}}$, where $V_i^{\text{adj}} \subseteq V$ denotes the set of vertices adjacent to $i$. When considering moving a boundary vertex $i$, the gain calculation depends on the relationship that $i$ and each of its neighboring vertices have with other partitions. The effect on volume depends on which partitions they share. In particular, we define for every vertex $i$ and every neighbor vertex $v$

$$N_{i-v} = N_i \setminus (N_v \cup P[v]) \quad \text{and} \quad N_{iv} = (N_i \cap N_v) \cup P[v] \tag{4.2}$$

representing partitions exclusive to $i$ and partitions shared by $i$ and $v$, respectively. In both cases, we add in $P[v]$ as this partition is shared even though it is not part of the $N_v$ set. A vertex $v$ has access to its own partition $P[v]$, meaning that $P[v]$ needs to be removed in $N_{i-v}$ and added in $N_{iv}$ to precisely determine shared partitions. Notice how both sets are subsets of $N_i = N_{i-v} \cup N_{iv}$.

As a concrete example, consider Figure 4.3 and suppose the candidate vertex is $A$, with neighbor $B$ under consideration. Here, $N_A = \{\text{blue}, \text{green}\}$, while $N_B = \{\text{blue}\}$. Consequently, $N_{A-B} = \{\text{green}\}$, since green is the only partition connected to $A$ but not to $B$, whereas $N_{AB} = \{\text{blue}\}$ as both vertices have at least one edge to the blue partition. Keeping $A$ as the candidate vertex, consider now neighbor $C$ with $N_C = \{\text{red}, \text{green}\}$. $A$ and $C$ have access to the same partitions, meaning that $N_{A-C} = \emptyset$ and $N_{AC} = \{\text{blue}, \text{green}\}$. This example highlights the need for adding $P[C]$ to the sets. The sets would have been $N_{A-C} = \{\text{blue}\}$, $N_{AC} = \{\text{green}\}$ if we had not accounted for $P[C]$. However, by simply viewing Figure 4.3 we can see that the blue partition is shared and not unique to $A$.

Let $C_i[n]$ denote the number of edges connecting $i$ to partition $n$. In Figure 4.3, for example, $C_A[\text{blue}] = 1$, as $A$ has only one edge connected to the blue partition, while $C_D[\text{blue}] = 2$, as $D$ has two edges connected to the blue partition. Let $G_i$ be a vector with a number of elements equal to the number of partitions, initialized at 0, where $G_i[n]$

represents the total gain obtained by moving vertex $i$ to partition $n$. For a candidate vertex $i$ being considered for a move, the gain contributions to all possible destination partitions are evaluated for each neighbor $v \in V_i^{\text{adj}}$ as follows:

**Case 1.** If $P[v] = P[i]$, then moving $i$ to any partition in $k \in N_{i-v}$ creates a shared inter-partition connection for $v$, yielding

$$G_i[k] -= w(i) \tag{4.3}$$

**Case 2.**If $P[v] \neq P[i]$ and $C_v[P[i]] = 1$, then moving $i$ to any partition $k \in N_{iv}$ eliminates an inter-partition connection for $v$, yielding

$$G_i[k] += w(i) \tag{4.4}$$

**Case 3.**If $P[v] \neq P[i]$ and $C_v[P[i]] > 1$, then moving $i$ to any partition $k \in N_{i-v}$ introduces a new inter-partition connection for $v$, resulting in a gain contribution of

$$G_i[k] -= w(i) \tag{4.5}$$

**Case 4.**If $C_i[P[i]] = 0$, i.e., $i$ has no internal degree, then moving $i$ to any partition $k \in N_i$ yields an additional gain contribution of

$$G_i[k] += w(i) \tag{4.6}$$

For every adjacent boundary vertex $v$ visited, we check each of the cases above for every partition in $N_{iv}$, $N_{i-v}$ or $N_i$, depending on the case. These cases represent all possible changes in communication volume when moving a vertex. Algorithm 1 in Section 4.5 shows how the gain is calculated for every vertex $i$ being considered moved. In turn, Section 5.1 shows how the gain vector $G$ is used during refinement to decide vertex moves.

## 4.3 Partition specific gain calculation

This section extends the total communication volume gain calculation from Section 4.2. The total gain contains no information about which partition is affected by a move, information that is required to use NVOL during refinement. Here we show how to decompose the communication volume gain into its individual contributions to each partition. Once computed, this decomposition can be used to refine the graph; how NVOL uses it is discussed in Section 5.2.

In order to calculate the volume for a given partition $n$, consider a subset of boundary vertices where each vertex is part of the same partition. The outgoing communication volume associated with partition $n$ is then given by

$$\text{partition} \quad \text{volume}[n] = \sum_{v \in V_b | P[v]=n} |N_v| \, w(v) \tag{4.7}$$

We introduce a table $SG_i$, where each row corresponds to a source partition and each column to a destination partition for vertex $i$. The entry $SG_i[n][k]$ represents the gain contribution to partition $n$ when $i$ is moved to partition $k$. In a graph with $m$ partitions, a vertex $i$ to partition $n$ the total gain satisfies

$$\sum_{t=0}^{m-1} SG_i[t][n] = G_i[n] \tag{4.8}$$

i.e., the sum of each column in $SG_i$ equals the corresponding entry in $G_i$. For a candidate vertex $i$ being considered for a move, the partition-specific gain contributions to all possible destination partitions are evaluated for each neighbor $v \in V_i^{\text{adj}}$ as follows:

**Case 1.** If $P[v] = P[i]$, then moving $i$ to any partition $k \in N_{i-v}$ removes a shared inter-partition connection. This affects the source partition:

$$SG_i[P[i]][k] -= w(i) \tag{4.9}$$

**Case 2.** If $P[v] \neq P[i]$ and $C_v[P[i]] = 1$, then moving $i$ to any partition $k \in N_{iv}$ eliminates an inter-partition connection. This contributes to partition $P[v]$:

$$SG_i[P[v]][k] += w(i) \tag{4.10}$$

**Case 3.** If $P[v] \neq P[i]$ and $C_v[P[i]] > 1$, then moving $i$ to any partition $k \in N_{i-v}$, introduces a new inter-partition connection. This contributes negatively to partition $P[v]$:

$$SG_i[P[v]][k] -= w(i) \tag{4.11}$$

**Case 4.** If $C_i[P[i]] = 0$, then moving $i$ to any partition $k \in N_i$ yields an additional gain contribution for the destination partition:

$$SG_i[k][k] += w(i) \tag{4.12}$$

It is also necessary to account for how both the source and destination partitions are affected by a vertex move. Although such a move redistributes communication volume between partitions, it does not alter the total communication volume of the graph. As discussed previously, NVOL requires precise tracking of how each move impacts individual partitions. When vertex $i$ is moved to partition $k$, this induces an additional gain contribution given by

$$SG_i[P[i]][k] += |N_i| \cdot w(i), \tag{4.13}$$

$$SG_i[k][k] -= |N_i| \cdot w(i). \tag{4.14}$$

This is referred to as a global term operation, as it reflects the overall redistribution of communication volume between the source and destination partitions of vertex $i$, rather than interactions with individual neighbor vertices. Equation 4.13 represents the volume leaving the source partition, while Equation 4.14 the volume the destination receives. Consequently, it is computed once per vertex move, in contrast to the neighbor specific gain contributions described earlier, which are evaluated separately for each adjacent vertex.

Apart from this distinction, both strategies follow the same case structure; the principal difference lies in how they store the gain information. Rather than recording only the magnitude of the change, NVOL also tracks which partition is affected. As it will be shown in Section 5.2, the gain table is essential for enforcing the partition-specif logic used in NVOL, which permits moves that improves a single partition regardless of the total communication volume.

## 4.4 Examples

To better understand the definitions of communication volume and gain, we present a sequence of illustrative examples. Each example highlights a particular configuration and demonstrates how gain contributions arise under different vertex moves.

### 4.4.1 Two Partition example

Consider the graph in Figure 4.4, consisting of five vertices with unit weights. The graph is partitioned into two subsets: the red partition containing vertices $\{A, B, C\}$, and the blue partition containing vertices $\{D, E\}$. Using Equation 4.1, the total communication volume is 4.

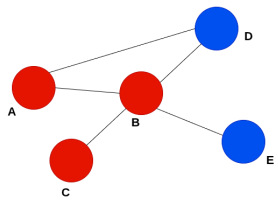


Figure 4.4: Partitioned graph with a volume of 4, each partition has an outgoing volume of 2.

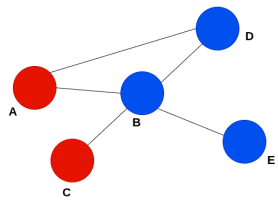


Figure 4.5: Partitinoed graph with a volume of 4, each partition has an outgoing volume of 2 after the move.

Vertex $A$ has one edge to the blue partition and contributes 1 to the communication volume. Vertex $B$ has two edges incident to the blue partition, but contributes only 1 under the given definition. Similarly, vertices $D$ and $E$ each contribute 1. In contrast, vertex $C$ contributes nothing, as all its incident edges are internal to its partition.

Now consider moving vertex $B$ from the red partition to the blue partition, as shown in Figure 4.5. We calculate the gain using the global strategy (Section 4.2). Let $B$ be the candidate vertex with neighbors $\{A, C, D, E\}$. Since there are two partitions, the gain vector $G_B$ has size two. In practice, however, $G_B$ effectively has size one, since $G_B[red]$ is always zero: a vertex moving from its source partition to the source partition is equivalent to no move at all.

**Vertex *A*.** Vertices $A$ and $B$ are in the same partition and both already have connections to the blue partition. Hence, the set $N_{B-A}$ is empty and no gain contribution arises.

**Vertex *C*.** Vertex $C$ lies in the red partition and has no edges to the blue partition. After moving $B$, $C$ gains an inter-partition edge, increasing the communication volume. As shown in Case 1, this yields a negative gain:

$$G_B[\text{blue}] -= 1.$$

**Vertex *D*.** Vertex $D$ is in the blue partition meaning that a move to the blue partition would decrease its volume only if one edge were incident on the red partition. Since $D$ has two, Case 3 does not apply, moving $B$ does not change its contribution, hence no case applies.

**Vertex *E*.** Vertex $E$ satisfies Case 2: it has exactly one connection to the red partition. After the move, this connection disappears, reducing the communication volume and yielding a positive gain:

$$G_B[\text{blue}] += 1.$$

Case 4 does not apply, as $B$ has multiple connections within its original partition. Summing all contributions yields total gain 0, so the communication volume remains unchanged.

**Partition-specific formulation.** Using the formulation from Section 4.3, we obtain the same result. We use the table $SG_B$ with size $2 \times 1$ (the source column will always be zeroed out just like the source in the gain vector), to store each volume change. As shown above only vertices $C$ and $E$ contribute:

$$SG_B[\text{red}][\text{blue}] -= 1, \quad SG_B[\text{blue}][\text{blue}] += 1.$$

Additionally, accounting for the direct effect of moving $B$ via Equations 4.13 and 4.14:

$$SG_B[\text{red}][\text{blue}] += 1, \quad SG_B[\text{blue}][\text{blue}] -= 1.$$

The net effect is zero, confirming that neither total nor per-partition volume changes.

### 4.4.2 Three partition example

We now extend the example to three partitions. Consider the graph in Figure 4.6, where all vertices have unit weight. The partitions are: red $\{A, B\}$, blue $\{C, D\}$, and green $\{E\}$. The total communication volume is 8, with partition volumes 3, 3, and 2, respectively.

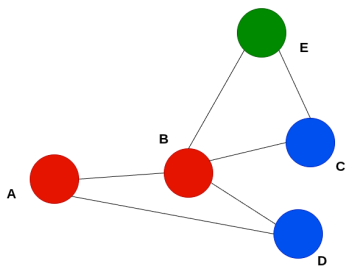


Figure 4.6: Partitioned graph with a total communication volume of 8.

Let $B$ be the candidate vertex with neighbors $\{A, C, D, E\}$. Unlike the previous example, $B$ may move to either the blue or green partition. Thus, $G_B$ has size 3 and $SG_B$ $3 \times 2$.

**Vertex *A*.** Vertex $A$ is in the same partition as $B$. Since $A$ is connected to the blue partition but not the green partition, Case 1 applies only for a move to green, introducing a new inter-partition edge:

$$G_B[\text{green}] -= 1.$$

**Vertex *C*.** Vertex $C$ is outside the red partition and has exactly one edge to it. By Case 2, removing this edge reduces the communication volume. Since $C$ is connected to both the blue and green partitions:

$$G_B[\text{blue}] += 1, \quad G_B[\text{green}] += 1.$$

**Vertex *D*.** Vertex $D$ has two edges to the red partition. By Case 3, a gain change occurs only when the destination partition is not already adjacent to $D$. This holds for the green partition:

$$G_B[\text{green}] -= 1.$$

**Vertex *E*.** Vertex $E$ is symmetrical to $C$, yielding:

$$G_B[\text{blue}] \mathrel{+}= 1, \quad G_B[\text{green}] \mathrel{+}= 1.$$

Summing contributions gives:

$$G_B = [0, 2, 0],$$

indicating that moving $B$ to the blue partition reduces the total communication volume by 2, while moving it to the green partition leaves it unchanged.

**Partition-specific formulation.** The vector $G_B$ alone does not capture how the volume is redistributed among individual partitions. Using Section 4.3, we compute $SG_B$ as follows.

**Vertex *A*.** Vertex $A$ follows Case 1for a move to the green partition, yielding:

$$SG_B[\text{red}][\text{green}] \mathrel{-}= 1$$

**Vertex *C*.** Vertex $C$ follows Case 2 for both moves, yielding:

$$SG_B[\text{blue}][\text{blue}] \mathrel{+}= 1$$
$$SG_B[\text{blue}][\text{green}] \mathrel{+}= 1$$

**Vertex *D*.** Vertex $D$ falls under Case 3 for the green partition, yielding:

$$SG_B[\text{blue}][\text{green}] \mathrel{-}= 1,$$

**Vertex *E*.** Vertex $E$ is symmetrical to $C$, yielding:

$$SG_B[\text{green}][\text{blue}] \mathrel{+}= 1$$
$$SG_B[\text{green}][\text{green}] \mathrel{+}= 1$$

Additionally, accounting for the direct effect of moving $B$ via Equations 4.13 and 4.14:

$$SG_B[\text{red}][\text{blue}] \mathrel{+}= 2, \quad SG_B[\text{red}][\text{green}] \mathrel{+}= 2,$$

$$SG_B[\text{blue}][\text{blue}] \mathrel{-}= 2, \quad SG_B[\text{green}][\text{green}] \mathrel{-}= 2.$$

The resulting table $SG_B$ is:

$$\begin{array}{c} \\ \text{red} \\ \text{blue} \\ \text{green} \end{array} \begin{array}{c} \begin{array}{cc} \text{blue} & \text{green} \end{array} \\ \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 1 & -1 \end{pmatrix} \end{array}$$

Each column corresponds to a destination partition, and each row describes how a partition's volume changes. One can verify that the column sums recover $G_B$: the sums are 0 (the red partition is implied as no move occurs if the vertex stays in the same partition it already part of), $2 + (-1) + 1 = 2$ and $1 + 0 + (-1) = 0$ , consistent with $G_B = [0, 2, 0]$. This formulation provides finer insight: although moving $B$ to the green partition leaves the total volume unchanged, it redistributes volume across partitions. In particular, the blue partition experiences an increase in volume at the partition level.

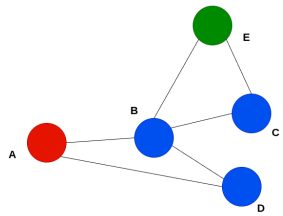


Figure 4.7: Partitioned graph has a volume of 6 after $B$ is moved to the blue partition

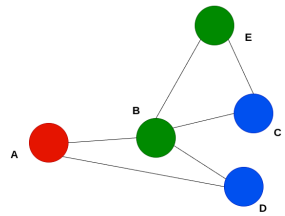


Figure 4.8: Partitioned graph has a volume of 8 after $B$ is moved to the green partition

### 4.4.3 Case 4 example

None of the preceding examples illustrate a scenario in which Case 4 is relevant. To clarify how such a case may arise and how it affects the communication volume, consider Figure 4.9, where all vertices carry unit weight. The partition assignment is as follows: red $\{A, B, C, G\}$, blue $\{D, E\}$, and green $\{F\}$. The total communication volume is 8, with individual partition volumes of 3, 3, and 2, respectively. As in the previous example, let $G_C$ have size 3 and $SG_C$ size $3 \times 3$.

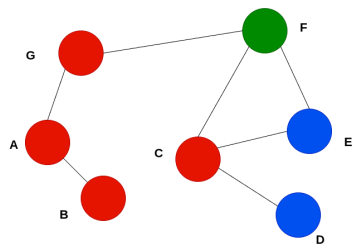


Figure 4.9: Partitioned graph with a total communication volume of 8.

The candidate vertex is $C$, which has no internal edges within the red partition. Despite the graph containing additional vertices, only the direct neighbors $\{D, E, F\}$ of $C$ need to be visited in order to compute the gain. This illustrates how, even in large graphs, gain computation remains local to the immediate neighborhood of the moving vertex. For conciseness, both computation strategies are consolidated in this example, as they have been thoroughly demonstrated in earlier sections.

**Vertex *D*.** Vertex $D$ satisfies Case 2 for a move to the blue partition, yielding:

$$\begin{aligned} G_C[\text{blue}] &\mathrel{+}= 1 \\ SG_C[\text{blue}][\text{blue}] &\mathrel{+}= 1 \end{aligned}$$

**Vertex *E*.** Vertex $E$ satisfies Case 2 for all possible moves of $C$, yielding:

$$\begin{aligned} G_C[\text{blue}] &\mathrel{+}= 1 \\ G_C[\text{green}] &\mathrel{+}= 1 \\ SG_C[\text{blue}][\text{blue}] &\mathrel{+}= 1 \\ SG_C[\text{blue}][\text{green}] &\mathrel{+}= 1 \end{aligned}$$

**Vertex *F*.** Vertex $F$ has two edges connecting it to the red partition; however, the set $N_{C-F}$ is empty, meaning Case 3 does not apply and vertex $F$ contributes no change to the gain

**Case 4.** Since vertex $C$ has no internal edges, any move will reduce the volume of its source partition. Under normal circumstances, when the candidate vertex has at least one internal edge, moving it leaves the receiving partition's incoming volume unchanged through the candidate. In this case, however, it is necessary to increment the gain for every possible destination partition, yielding:

$$\begin{aligned} G_C[\text{blue}] &+= 1 \\ G_C[\text{green}] &+= 1 \\ SG_C[\text{blue}][\text{blue}] &+= 1 \\ SG_C[\text{green}][\text{green}] &+= 1 \end{aligned}$$

The resulting vector $G_C = [0\,3\,2]$. Accounting for the direct effect of relocating $C$ via Equations 4.13 and 4.14:

$$SG_C[\text{red}][\text{blue}] += 2, \quad SG_C[\text{red}][\text{green}] += 2,$$

$$SG_C[\text{blue}][\text{blue}] -= 2, \quad SG_C[\text{green}][\text{green}] -= 2.$$

The resulting table $SG_C$ is:

$$\begin{array}{c} \\ \text{red} \\ \text{blue} \\ \text{green} \end{array} \begin{array}{c} \begin{array}{cc} \text{blue} & \text{green} \end{array} \\ \begin{pmatrix} 2 & 2 \\ 1 & 1 \\ 0 & -1 \end{pmatrix} \end{array}$$

Figures 4.10 and 4.11 show the resulting graphs after moving $C$ to the blue and green partitions respectively. The reader can verify that the partition volumes in each case are consistent with the gains predicted by $SG_C$ and $G_C$. Without accounting for Case 4, the gain calculation would be incomplete, potentially leading to incorrect volume estimates when a vertex has no internal edges.

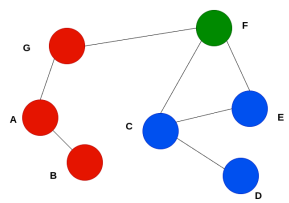


Figure 4.10: Partitioned graph has a volume of 5 after the move.

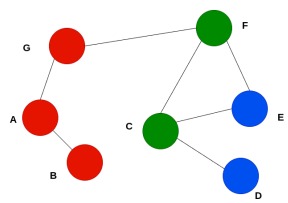


Figure 4.11: Partitioned graph has a volume of 6 after the move.

## 4.5 Implementation

We now describe how the calculation of communication volume change is integrated into the multilevel framework. As mentioned previously, we have used METIS as the base on which to develop NVOL. The following description bridges the formal definitions above and the implemented code.

### 4.5.1 Total gain implementation

In Section 3.4.1 the boundary set is computed using edge cut gain to filter and order it. When using total communication volume, we need to do the same thing using the calculations described in this chapter. Unlike the edge-cut metric, we cannot state for a given vertex whether moving it will lead to a positive gain or not. As stated in Section 3.4.1, we can deduce whether moving a vertex is guaranteed to give negative gain. This is not transferable to communication volume metrics.

With communication volume, it is necessary to visit all neighbors of a vertex to correctly evaluate the gains of possible moves. This makes the calculation difficult to integrate directly into the preparation phase before refinement. In METIS, this issue was resolved by adding the gain calculation as a final step after preparation. Once the graph has been projected and all underlying structures have been recomputed, a full scan of the graph is performed to evaluate the possible gains for every vertex.

**Algorithm 1** Total gain calculation

```
nparts ← k                                          ▷ Graph with k partitions
for all i ∈ V do
    if |N_i| > 0 then                               ▷ Only visit vertices with neighboring partitions
        for all v ∈ V_i^adj do                      ▷ Visit all adjacent vertices of i
            for n = 0; n < nparts do
                if n ∈ N_i AND n ∈ N_v then
                    N_iv.add(n)                     ▷ Shared partitions between i and v (N_iv)
                else if n ∈ N_i AND n ∉ N_v then
                    N_{i−v}.add(n)                  ▷ Partitions only i is connected to (N_{i−v})
                end if
            end for
            if P[i] = P[v] then
                for all n ∈ N_{i−v} do
                    G[n] −= w(i)                    ▷ Case 1
                end for
            else if P[i] != P[v] AND C_v[P[i]] = 1 then
                for all n ∈ N_iv do
                    G[n] += w(i)                    ▷ Case 2
                end for
            else if P[i] != P[v] AND C_v[P[i]] > 1 then
                for all n ∈ N_{i−v} do
                    G[n] −= w(i)                    ▷ Case 3
                end for
            end if
        end for
        if C_i[P[i]] = 0 then
            for all n ∈ N_i do
                G[n] += w(i)                        ▷ Case 4
            end for
        end if
        BoundarySet.add(MAX(G_v))                   ▷ Order by highest possible gain
    end if
end for
```

### 4.5.2 Partition specific gain implementation

The implementation of the partition-specific gain calculation required a different approach, even though the logic in the inner loop is very similar. Before starting refinement, METIS calculates and stores the gain of all boundary vertices. However, NVOL requires significantly more memory: for a gain vector of 10 elements, NVOL's gain table contains 90 entries. For this reason, we opted for an on-the-fly approach, computing the gain table for each vertex as it is visited during refinement. Beyond the memory overhead, a precomputed gain table would also require an update procedure to maintain consistency after each move. As this is not directly relevant to the research questions of this thesis, we leave this as future work, discussed further in Chapter 8.

NVOL is implemented directly in the refinement stage. During preparation, it uses the same structure as Section 4.5.1, ordering vertices based on their total gain. Once refinement begins, the algorithm visits every vertex in the boundary set. For each vertex $i$ under consideration, we run the algorithm below:

**Algorithm 2** Partition specific gain calculation

```
nparts ← k                                          ▷ Graph with k partitions
for all v ∈ V_i^adj do                              ▷ Visit all adjacent vertices of i
    for n = 0; n < nparts do
        if n ∈ N_i AND n ∈ N_v then
            N_iv.add(n)                             ▷ Shared partitions between i and v (N_iv)
        else if n ∈ N_i AND n ∉ N_v then
            N_i−v.add(n)                            ▷ Partitions only i is connected to (N_i−v)
        end if
    end for
    if P[i] = P[v] then
        for all n ∈ N_i−v do
            SG[P[v]][n] −= w(i)                     ▷ Case 1
        end for
    else if P[i] != P[v] AND C_v[P[i]] = 1 then
        for all n ∈ N_iv do
            SG[P[v]][n] += w(i)                     ▷ Case 2
        end for
    else if P[i] != P[v] AND C_v[P[i]] > 1 then
        for all n ∈ N_i−v do
            SG[P[v]][n] −= w(i)                     ▷ Case 3
        end for
    end if
end for
if C_i[P[i]] = 0 then
    for all n ∈ N_i do
        SG[n][n] += w(i)                            ▷ Case 4
    end for
end if
```

# Chapter 5

# Refinement Using Communication Volume

This chapter will discuss how the actual refinement is performed. Using the gains calculated in the previous chapter, we will explain how a vertex move is decided. This chapter will also compare how the choice is made using a total communication volume objective function and the partition-aware communication volume used in NVOL.

## 5.1 Refinement using total communication volume

After the preparation phase is completed, refinement starts and the partitioner visits all the vertices in the boundary set. For each vertex $i$, all the neighboring partitions of $i$ are visited, checking the values of the $G_i$ vector. A vertex move is chosen if it improves the total communication volume while maintaining the weight constraints, just like for GR in Section 3.4.2. The refinement algorithm will look for the move with the highest value in $G_i$ that is allowed by the balance constraints. To prioritize high gain moves, METIS relaxes the balance constraint in proportion to the gain. Using a constant, called the fudge factor $f$, a move to partition $p$ is allowed to exceed the maximum partition weight by $f \cdot G_i[p]$. This allows moves that improve the communication volume significantly at the cost of a few extra vertices in a partition.

When two moves have the same gain, METIS breaks ties using two further criteria: external degree and partition balance. In the case of a tie, it first chooses the move with the highest specific external degree (see Section 3.4.1). Since all vertices within a coarse vertex are assigned to the same partition upon projection, a move toward a partition with high external degree may reduce volume at finer levels. If instead two candidates have the same gain and the same external degree, the move to the more underweight partition is chosen.

Consider a vertex $i$ being visited in the boundary set during refinement. Algorithm 3 shows how METIS chooses vertex moves using the logic described above.

**Algorithm 3** Total communication volume refinement

```
Require: Vertex i with neighbor partitions N_i[0 ... n−1]
  if w(P[i]) − w(i) ≥ W^min then
      for k = n−1 downto 0 do                          ▷ Find first feasible candidate
          if G_i[N_i[k]] ≥ 0 AND
             w(N_i[k]) + w(i) ≤ W^max + f · G_i[N_i[k]] then
              break
          end if
      end for
      if k < 0 then continue to next vertex
      end if
      for j = k−1 downto 0 do                          ▷ Search for a better candidate
          p_j ← N_i[j], p_k ← N_i[k]
          if G_i(p_j) > G_i(p_k) AND
             w(p_j) + w(i) ≤ W^max + f · (G_i(p_j)) then
              k ← j                                    ▷ Better gain, relaxed balance
          else if G_i(p_j) = G_i(p_k) AND
                  ED_{p_j}[i] > ED_{p_k}[i] AND
                  w(p_j) + w(i) ≤ W^max then
              k ← j                                    ▷ Same gain, better connectivity
          else if G_i(p_j) = G_i(p_k) AND
                  ED_{p_j}[i] = ED_{p_k}[i] AND
                  w(p_j) < w(p_k) then
              k ← j                                    ▷ All else equal, prefer underloaded partition
          end if
      end for
      Move i to N_i[k]
  end if
```

## 5.2 Refinement using partition-specific communication volume

We now describe how refinement is performed using the partition-specific communication volume objective in NVOL. In Section 4.5.2 we explained how, for a boundary vertex $i$, the gain table $SG_i$ is computed. To briefly recall, each entry $SG_i[p][j]$ records the change in outgoing communication volume that partition $p$ would experience if vertex $i$ were moved to neighboring partition $j$. The rows of the table correspond to affected partitions, while the columns correspond to possible destinations. The goal of NVOL is to minimize the maximum outgoing communication volume held by any single partition.

### 5.2.1 Move selection

During refinement, NVOL evaluates every candidate destination for a boundary vertex by inspecting the gain table. For each potential move of vertex $i$ to a destination partition $j$, NVOL determines what the highest volume among all affected partitions would be after the move. If any partition would exceed the current global maximum partition volume, the highest volume held by any partition in the entire graph, the move is immediately discarded. As with the total communication volume approach, NVOL also enforces partition weight constraints: any move must keep the destination partition's weight within the allowed bounds.

NVOL ranks candidate moves according to three criteria, applied in the following order of priority: maximum local partition volume, external degree and balance. The first criterion seeks a move that minimizes the local top volume. Out of all the affected partitions, NVOL will choose the move that decreases the top volume of these partitions the most, as long as that top volume is lower than the global maximum. The second criterion is used when two moves have the same local top volume. As a tiebreaker, NVOL favors destinations toward which the considered vertex has a high external degree. This follows the same logic as the total communication volume approach, aiding moves where the vertex is strongly connected. The final criterion is used when the previous two are tied. NVOL prefers moving vertices to lighter partitions to better balance the weight distribution between the partitions.

### 5.2.2 Encouraging more moves

A deliberate design choice in NVOL was to promote many vertex moves. Especially when choosing the first candidate, NVOL does not require an immediate improvement of the local top volume. Instead, it chooses any move that leads to a local top that is below the global maximum. This means that in practice NVOL might choose moves that increase the local top volume as long as it is below the global maximum.

The goal with this design is to increase the number of moves NVOL performs during a refinement stage. During testing, we experimented with a stricter criterion that required every accepted move to improve the local top volume. Under this stricter rule, significantly fewer vertices were moved during refinement. On average, the stricter rule performed half as many moves as the more relaxed approach. By encouraging more moves, we encountered two effects. First, NVOL was able to explore more solutions: since more moves were performed, more setups were evaluated. Second, we increased the chance of avoiding getting stuck in a local maximum. If no move could possibly improve the partition's volume, it would remain stuck; by allowing a "worsening" move, we increase the chance of finding better solutions. Since the condition still guarantees that the global maximum is never exceeded, the objective value is protected throughout the process.

To prevent this relaxed condition from accepting moves that are too damaging, NVOL incorporates the fudge factor as a safety mechanism. The fudge factor is used to adjust the maximum allowed partition weight; however, in NVOL we multiply it by the gain for the top partition. Specifically, for a vertex $i$, a partition $m$ with the highest local volume, and a destination partition $j$, we multiply the fudge factor by $SG_i[m][j]$. This ensures that moves that have a negative impact on the local top partition are more likely to be filtered out by the weight constraint. By making the maximum allowed weight smaller, it gets harder to accept a move. In the opposite case, it works as a relaxing factor, allowing moves that positively improve the local top. In this way, $f$ acts as a soft penalty that discourages harmful moves without outright forbidding all non-improving ones. In contrast to the total volume approach, this is used for all criteria other than balance.

### 5.2.3 Implementation of refinement

For a boundary vertex $i$, NVOL iterates over the neighboring partitions $N_i$. For each candidate destination $j$, it first determines which of the affected partitions would hold the highest volume after the move. In order to do that, it scans all entries $SG_i[k][j]$ for $k \in N_i$ and computes $PV[k] + SG_i[k][j]$, where $PV[k]$ is the volume partition $k$ holds

before the move. The partition achieving the highest post-move volume is recorded as the local top partition, with volume tmpVol and partition id tmpPid.

If no candidate has been found yet, NVOL applies the relaxed first condition: destination $j$ is accepted as a candidate if tmpVol does not exceed the global maximum partition volume topVol, and the weight constraint $w(j) + w(i) \leq W^{\max} + f \cdot SG_i[\text{tmpPid}][j]$ is satisfied.

Once a first candidate has been found, NVOL tries to find a better destination. A candidate is replaced if the new destination leads to a lower local top volume, using the same weight constraint as above. In the case of a tie, destinations with high external degree $ED_j[i]$ are preferred. If both local top volume and external degree are tied, the destination with the lower current weight is chosen, favoring underloaded partitions. After all neighboring partitions have been considered, vertex $i$ is moved to the final selected destination. The full procedure is given in Algorithm 4.

**Algorithm 4** NVOL refinement

```
Require: Vertex i with neighbor partitions N_i[0...n−1];Max partition volume topVol
  if w(P[i]) − w(i) ≥ W^min then
      candidateVol ← −1
      destination ← −1
      for all j ∈ N_i do
          tmpVol ← 0
          tmpPid ← −1
          for all k ∈ N_i do
              if tmpVol< PV[k] + SG_i[k][j] then
                  tmpVol ← PV[k] + SG_i[k][j]
                  tmpPid ← k
              end if
          end for
          if candidateVol = -1 AND
             topVol ≥ tmpVol AND
             w(j) + w(i) ≤ W^max + f · SG_i[tmpPid][j] then
              destination ← j            ▷ Move does not increase max partition volume
              candidateVol ← tmpVol
          else if candidateVol != -1 AND
                  candidateVol > tmpVol AND
                  w(j) + w(i) ≤ W^max + f · SG_i[tmpPid][j] then
              destination ← j            ▷ Lower local max partition volume after move
              candidateVol ← tmpVol
          else if candidateVol != -1 AND
                  candidateVol = tmpVol AND
                  ED_j[i] > ED_destination[i] AND
                  w(j) + w(i) ≤ W^max + f · SG_i[tmpPid][j] then
              destination ← j            ▷ Same gain, better connectivity
              candidateVol ← tmpVol
          else if candidateVol != -1 AND
                  candidateVol = tmpVol AND
                  ED_j[i] = ED_destination[i] AND
                  w(j) < w(destination) then
              destination ← j            ▷ All else equal, prefer underloaded partition
              candidateVol ← tmpVol
          end if
      end for
      Move i to destination
  end if
```

# Chapter 6

# METIS and Supporting Tooling

This chapter presents the tools developed and used throughout this thesis to produce the results in Chapter 7. The first section describes in detail how METIS was adapted to support NVOL, while the second section covers the tools used for data analysis and visualization.

## 6.1 METIS

For this thesis we used METIS as our multilevel $k$-way partitioner. The METIS framework is open source[1], developed by George Karypis, and widely used. We have also made public a version extended with NVOL[2]. Most of the logic behind the multilevel $k$-way algorithm has already been covered in Chapters 3-5, so we refer the reader to those chapters.

### 6.1.1 Implementing NVOL

Implementing NVOL required extending METIS's existing infrastructure. We first defined a new volume-based objective named NVOL. Because it is volume-based, NVOL reuses the coarsening and initial partitioning code of the total volume objective; our extensions are confined to the refinement stage.

We added two new structs to METIS: one to carry per-partition information, and one for the refinement table. The `pinfo_t` struct holds an integer representing a partition's outgoing volume, and METIS's main `graph_t` struct now carries a pointer to it. At the start of refinement we allocate an array of $k$ `pinfo_t` entries, one per partition, and update them throughout refinement to track partition-specific volumes. The second struct, `vol_refinement_table`, holds a pointer to an integer array; `graph_t` instead stores a pointer to a `vol_refinement_table` struct, yielding effectively an array of `vol_refinement_table`. This forms the gain table described in Chapter 4.

When refinement begins, METIS allocates all of the memory needed by its refinement structures. NVOL reuses these structures and adds the two described above. The per-partition info requires $k$ integers, while the refinement table requires $k \times k$. Allocating a full table for every boundary vertex would be prohibitively expensive, so we instead allocate a single shared refinement table that is repopulated for each vertex under consideration. This keeps NVOL's memory overhead only marginally higher than METIS's.

[1] https://github.com/KarypisLab/METIS
[2] https://github.com/riccardoie/METIS-with-NVOL

During the preparation phase at each level, METIS clears its refinement structures and recomputes them. The graph is projected up to a finer level, and all the structs containing information about neighboring vertices, partitions, and edges need to be updated. We added NVOL as a case in all relevant functions. Most of the existing code was reusable, since NVOL shares the same base as the total volume objective; we extended it only to populate `pinfo_t`. While evaluating the external degree and the set of neighboring partitions of each vertex, we now also store this information in `pinfo_t`. The computational cost of this operation is negligible as the information was already being computed, just not retained. For each vertex visited, we increase its partition's outgoing volume by the vertex size for every unique neighbor partition it has.

The heart of NVOL lives in `kwayfm.c`, which contains all of the refinement algorithms. The file's dispatching function, `Greedy_KWayOptimize`, selects a refinement algorithm based on the objective; we added a new branch that dispatches to our refinement function, `Greedy_KWayNVolOptimize`. This function iterates over the boundary set and decides which vertex moves to perform. The NVOL refinement implementation rests on two functions: `Greedy_KWayNVolOptimize` and `compute_potential_move_volume_change`. The former is adapted from METIS's existing `Greedy_KWayVolOptimize`, retaining the overall priority-queue-driven structure but replacing the move-selection logic to target the NVOL objective. The latter is a new helper that supplies the per-partition volume estimates required by the modified selection logic.

### Greedy_KWayNVolOptimize

`Greedy_KWayNVolOptimize` follows the same outer structure as the other greedy refinement algorithms: boundary vertices are inserted into a priority queue ordered by their total communication volume gain. If we wanted to use partition-specific gain here instead we would have had to calculate the refinement table during preparation and then later recalculate it during refinement. To avoid repeating this operation we settled for using the total gain as a measure, as it was ready to use and it equals the sum of partition-specific gains. Vertices are then extracted one at a time and considered for relocation. NVOL diverges from METIS only in how it chooses the destination partition. This selection is detailed in algorithm 4.

Each time a vertex is extracted, NVOL populates the gain calculation table by using `compute_potential_move_volume_change` (described below). It then iterates over all possible destinations to find the most viable option. Each iteration follows the same pattern: identify the highest local volume that would result from the move, then evaluate the move. Because only the neighbors of the extracted vertex experience a change in volume, NVOL inspects only those vertices via an inner loop. There, `pinfo_t` provides the current partition volumes, from which the corresponding gain values in the gain table are subtracted.

After the inner loop, NVOL evaluates the candidate move through a series of conditional checks. If no move has yet been accepted, we simply require that the highest local volume after the move be smaller than the highest recorded volume in the graph. This value is stored in the `graph_t` struct, and updated in `compute_potential_move_volume_change`. By doing this, we ensure that any accepted move will not increase the graph's maximum per-partition outgoing volume. Once an initial candidate has been accepted, we look for the move that yields the smallest maximum local volume. Ties are broken in two stages: we first prefer destinations towards which the vertex has greater external degree, and if that is also tied we prefer

moves toward the more underweight partition. If no candidate is found, NVOL goes on to the next vertex.

Once every destination has been evaluated and a move selected, we update the `pinfo_t` information. The remaining machinery (boundary updates, subdomain connectivity tracking, and the rebalancing branch) is left essentially unchanged from the upstream METIS code. This operation will be repeated until the priority queue has been emptied. The greedy refinement is repeated a set number of times, however, if no moves are made or no change in volume occurs, the algorithm exits early; NVOL inherits this behavior.

#### compute_potential_move_volume_change

`compute_potential_move_volume_change` is the helper that makes the selection of moves possible. It implements the algorithm described in Section 4.5.2. The function begins by zeroing out the refinement table to ensure no stale values from other vertices remain. Since this pass already touches each of the $k$ rows of the table, we use the opportunity to also record which partition currently has the highest volume; this value defines the ceiling when choosing moves. The double-pointer structure of the refinement table behaves as a matrix: the first index identifies a row and the second a column, so `vol_ref_table[p].gain_list[j]` stores the volume change that partition $p$ would experience if vertex $i$ were moved to partition $j$. To fill this table, the function walks the neighborhood of $i$ and updates the relevant entries according to the first three cases described in Section 4.3. Once all neighbors have been accounted for, the function handles case 4 and the volume contribution of the moving vertex through an additional loop, adjusting the volumes of the source and destination partitions accordingly.

#### Time complexity and efficiency

### 6.1.2 Inputs

METIS requires three inputs: a graph file, the number of partitions, and the objective function. A few optional flags are also available to fine-tune the algorithm.

The graph file must follow a specific structure. The first line is a header whose first two values are the number of vertices and the number of unique edges in the graph. The header may also contain two additional parameters specifying whether the graph has vertex/edge weights and whether it has multiple weights per vertex or edge. Further details can be found in the official manual[3]. All graphs used in this work are single-weight, with unit weights on both edges and vertices, a list of them can be found in table 7.1 and table 7.2. Note, however, that NVOL also supports graphs with single vertex weights.

The remaining lines of the graph file describe the graph's connectivity: each line corresponds to a vertex, and its entries are the ids of the vertices it is connected to.

### 6.1.3 Output

Once the algorithm terminates, METIS produces a partition file. This text file contains $n$ lines, where $n$ is the number of vertices in the graph, and each line holds a single integer in the range $[0, k-1]$ indicating the partition the corresponding vertex was assigned to. The partition files are name graphName.part.$k$, where $k$ is the number of partitions.

[3] https://github.com/KarypisLab/METIS/blob/master/manual/manual.pdf

## 6.2 Graph Tools

In this section we will present a few simple tools that were developed during this thesis to display and analyze the results produced by the partitioning algorithm as well as graph formatting. The scripts can all be found in this public repository [4].

### 6.2.1 calc_partitions

To verify and analyze the per-partition outgoing and incoming communication volumes produced by our experiments, we wrote a small program, `calc_partitions`, that computes them directly from the partition files. The program contains two functions, `outgoing_volstats` and `incoming_volstats`, each taking two arguments: the original graph filename and the path to the folder containing the partition files. Given these inputs, each function computes its respective volume for every `.part.`$k$ file associated with that graph. The results are written to a text file divided into sections, one per partition file, each introduced by a header of the form `graphName.part.`$k$. Within each section, the partitions are listed in order of partition id, from 0 to $k-1$. Each function produces its own output file.

The outgoing communication volume of a partition was defined in Section 4.3 as the sum of the outgoing volumes of the vertices it contains. The incoming volume has not been introduced yet, so we extend the definitions accordingly. If outgoing communication volume measures the data a partition must send out, incoming communication volume measures the data it must receive from outside. Using the notation from Chapter 4, the incoming communication volume of partition $n$ is

$$\text{incoming_volume}[n] = \sum_{\substack{v \in V_b \\ n \in N_v}} w(v), \tag{6.1}$$

where $V_b$ is the set of boundary vertices and $N_v$ is the set of partitions of $v$'s neighbors, excluding $v$'s own partition $P[v]$. In other words, every boundary vertex $v$ contributes its weight $w(v)$ to each partition that are connected to it.

Both functions share the same traversal. For every partition file, the function initializes an array of length $k$ and then iterates over the graph's vertices, collecting for each vertex the set of distinct partitions among its neighbors that differ from its own. The two functions only diverge in how they update the array from this set: `outgoing_volstats` adds the size of the set to the entry for the vertex's own partition, while `incoming_volstats` increments by one the entry of every partition in the set. Using a set ensures each external partition is counted at most once per vertex, matching the definition of communication volume from Section 4.3. Once the graph has been fully traversed, the resulting array is stored in a dictionary keyed by `graphName.part.`$k$ and written to the output file.

### 6.2.2 form_statics

To collect the per-partition statistics produced by our experiments without recomputing them, we use the script `form_statistics`, which processes the files produced by `calc_partitions`. For each file, the script reads the contents line by line and parses two types of entries: lines containing `.part.`, treated as headers marking the beginning of a new partition file's section, and lines starting with `Pid`, containing the volume of the

[4] https://github.com/riccardoie/Graph-tools/tree/main

corresponding partition. The values are grouped by section in a dictionary, and for each group we compute the total outgoing communication volume, the maximum outgoing volume across partitions, the average outgoing volume, and the standard deviation. The results are written to a CSV file, one per input file.

### 6.2.3 graph_results_vol

To evaluate the distribution, we have developed a simple script that plots the partition volumes for a given partition file. The script takes three inputs: the graph file, the number of partitions, and the version identifying the partition file folder. It reads the files output by `calc_partitions` and collects all values from the section matching the requested number of partitions. The script is also hardcoded to do the same for the corresponding METIS volumes, and the two sets of volumes are then plotted side by side.

### 6.2.4 stats_vol

This script reads the files produced by `form_statistics` and plots the results. It takes a graph file name and the version being tested as input, then compares the resulting statistics against the METIS reference values. It produces four plots: the total volume, the maximum partition volume, the percentage difference in maximum partition volume between NVOL and METIS, and the standard deviation of the partition volumes.

### 6.2.5 translate_to_metis

All graphs used in this thesis were in Matrix Market exchange format (.mtx) and therefore needed to be translated into a format METIS could use. The .mtx format is a plain-text format for representing sparse and dense matrices, where each non-zero entry corresponds to an edge in the graph. After the comment lines (prefixed with %) and a header, each subsequent line lists the two endpoints of an edge: for example, 1 24 denotes an edge between vertex 1 and vertex 24. Our script parses the file and stores each edge as a sorted tuple in a set, which deduplicates entries and discards self-loops. It then builds an adjacency list by iterating over the edge set, appending each vertex to each other's neighbor lists. Since Matrix Market files are 1-indexed while Python lists are 0-indexed, we offset the list indices by one while keeping the vertex ids themselves 1-indexed, as required by the METIS format. Once all edges have been processed, the adjacency list is written as a graph file.

### 6.2.6 model_estimates

To further evaluate the quality of our partitions, we implemented one of the models presented in [45]. These models estimate the per-process runtime of a communicating task while accounting for the bandwidth contention caused by competing MPI processes. We specifically use the model for processes with multiple neighbors and non-uniform message sizes, given by formulas 7-9.

Our script takes four arguments: the graph, the version of NVOL, the number of partitions, and the startup latency. The bandwidth array is currently hardcoded, but it could just as easily be exposed as an argument to adapt the model to other setups. Both the startup latency and the bandwidth values were provided by one of the authors of [45].

The script begins by computing the incoming communication volume for each partition. From these volumes, together with the bandwidth and latency parameters, we estimate how long each partition takes to complete its communication step. The formulas take the incoming volumes sorted from lowest to highest and evaluate the receive and send times for each partition: formula 7 estimates the total time a partition spends receiving messages as a function of its incoming volume, while formula 9 pinpoints the time to send a message to a specific destination.

Formula 8 then combines these results by taking the maximum of the receive time and the individual send completion times,a partition can only progress as fast as its slowest communication operation, and adds the startup latency multiplied by the number of neighboring partitions. This captures the fact that every message exchange incurs a fixed startup cost regardless of the volume transferred. All three formulas are applied to every partition, and the results are plotted as a bar graph.

$$T_i = M_i^{in} \cdot \tau + max\left(t_{i-1}^{\text{recv}}, t_{i,0}^{\text{send}}, t_{i,1}^{\text{send}}, \ldots, t_{i,M_i^{\text{out}}-1}^{\text{send}}\right) \tag{6.2}$$

# Chapter 7

# Results

This chapter presents the results of running METIS with NVOL. Throughout this chapter we will compare these results to partitionings using METIS's own total communication volume objective function. To generate the graphs and plots in this chapter, the tools described in Chapter 6 were used. These results show the impact of applying the partition-aware objective function NVOL and compare whether this leads to any improvements in the resulting partitioning.

The chapter follows this structure. The first section briefly describes the selection of graphs we have used for our tests. They are divided into two sets: the Heart dataset and the SuiteSparse Matrix Collection. The following section examines the maximum per-partition volume resulting from partitioning the graphs using NVOL and the total volume objective function. Here we also study how well the partitionings perform on other metrics, such as standard deviation, average volume, and so on. The third section shows how METIS can be tuned to affect refinement, and how NVOL reacts to this tuning, once again by looking at the partitioned graphs. In the final section we present the runtime estimates for the partitioned graphs. Here too, we analyze the results to understand why NVOL performs as it does.

## 7.1 Graphs used for testing

To evaluate NVOL we used symmetric matrices drawn from two datasets, treating each matrix as the adjacency representation of an undirected graph. A symmetric matrix is a matrix where element $a_{ij} = a_{ji}$, representing an undirected edge between $i$ and $j$. Following our definition of GPP in Section 2.1, we use therefore such matrices, it is however, possible to form graphs from other kinds of matrices as well. The two datasets were chosen to cover a wide range of graph sizes and to include instances arising from different application domains. We tried to prioritize matrices with fewer non-zero elements, or with a higher average degree (the degree represents the number of edges a vertex has).

The first dataset is taken from *Repository for modelling acute myocardial ischemia* [34], in which the authors simulate acute myocardial ischemia in a human heart using the Chaste finite-element framework. We simplify the name to Heart dataset. The matrices produced by the data in [34] were then translated into a format accepted by METIS. Table 7.1 contains the seven graphs, ranging from roughly $10^3$ to almost $10^7$ vertices. For each graph it reports the number of vertices, the number of edges, the maximum and minimum vertex degree, and the standard deviation of the vertex degree.

| Graphs | Vertices | Edges | Max degree | Min degree | Avg. degree | Std. dev. |
|---|---|---|---|---|---|---|
| heart01 | 1,394 | 6,946 | 21 | 5 | 10 | 2.1 |
| heart02 | 45,384 | 275,661 | 24 | 5 | 12 | 3.4 |
| heart03 | 296,774 | 1,948,530 | 25 | 5 | 13 | 3.1 |
| heart04 | 555,433 | 3,715,278 | 26 | 5 | 13 | 3.0 |
| heart05 | 1,257,588 | 8,574,893 | 25 | 5 | 14 | 2.8 |
| heart06 | 4,041,680 | 28,138,550 | 25 | 5 | 14 | 2.6 |
| heart07 | 9,349,543 | 65,800,809 | 26 | 4 | 14 | 2.6 |

Table 7.1: Heart dataset [34]

The second data set was retrieved from *The SuiteSparse Matrix Collection* [11]. This collection contains a large variety of matrices and keeps growing. We chose a subset of matrices that were symmetric with different numbers of vertices and with varying average degree. These matrices were then translated from Matrix Market to a format METIS supports. Table 7.2 summarizes the 13 matrices we chose ordered by vertex number.

| Graphs | Vertices | Edges | Max deg | Min deg | Avg. deg | Std. dev. |
|---|---|---|---|---|---|---|
| pa2010 | 421,545 | 1,029,231 | 64 | 1 | 4 | 2.6 |
| coPapersDBLP | 540,486 | 15,245,729 | 3,299 | 1 | 56 | 66.2 |
| packing -500x100x100 -b050 | 2,145,852 | 17,488,243 | 18 | 0 | 16 | 2.8 |
| channel -500x100x100 -b050 | 4,802,000 | 42,681,372 | 18 | 6 | 18 | 1.0 |
| hugetric-00000 | 5,824,554 | 8,733,523 | 3 | 2 | 3 | 0.03 |
| italy_osm | 6,686,493 | 7,013,978 | 9 | 1 | 2 | 0.4 |
| hugetric-00020 | 7,122,792 | 10,680,777 | 3 | 2 | 3 | 0.03 |
| rgg_n_2_23_s0 | 8,388,608 | 63,501,393 | 40 | 0 | 15 | 3.9 |
| delaunay_n23 | 8,388,608 | 25,165,784 | 28 | 3 | 6 | 1.3 |
| hugetrace-00020 | 16,002,413 | 23,998,813 | 3 | 2 | 3 | 0.02 |
| delaunay_n24 | 16,777,216 | 50,331,601 | 26 | 3 | 6 | 1.3 |
| kmer_V2a | 55,042,369 | 58,608,800 | 39 | 1 | 2 | 0.7 |
| kmer_U1a | 67,716,231 | 69,389,281 | 35 | 1 | 2 | 0.4 |

Table 7.2: Selection of matrices from SuiteSparse Matrix Collection [11]

Note that rgg_n_2_23_s0 and packing-500x100x100-b050 have a minimum degree of 0. That means that at least one of the vertices is isolated with no neighbors. We have purposely added two such graphs to test how NVOL performs on this type of graphs as well.

## 7.2 Maximum outgoing volume per partition

Using the graphs listed above, we ran METIS with two objective functions: NVOL (see Section 5.2) and VOL (see Section 5.1). For each run we then used `calc_partitions` to analyze the resulting partitions and compare their outgoing volumes. We partitioned the graphs into 2 to 2048 partitions; however, due to their size, not all graphs were partitioned up to 2048.

The following sections will go through the results from each dataset, specifically comparing the maximum outgoing partition volume ($W^{\max}$) between VOL and NVOL.

$W^{\max}$ represents the single highest outgoing volume of any partition in a partitioned graph. Using $W^{\max}_{\text{VOL}}$ as the baseline, we report each value as the percentage difference of $W^{\max}_{\text{NVOL}}$ relative to $W^{\max}_{\text{VOL}}$, $(W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100$. Cells are color-coded for readability: blue indicates that NVOL achieves a lower maximum (better), while orange indicates a higher maximum (worse). Shading intensity reflects the magnitude of the difference, with darker shades marking larger improvements or regressions. Some cells are marked with N/A, which simply means that the graph was too small for that number of partitions ($k$), cells marked N/R indicate runs that did not finish discussed further in Section 7.2.2.

### 7.2.1 Heart dataset

| **Graphs** | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 | 2048 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| heart01 | -1.30 | **1.04** | **14.40** | **6.49** | **0.00** | N/A | N/A | N/A | N/A | N/A | N/A |
| heart02 | **0.79** | **5.93** | **6.82** | **3.38** | **10.44** | **10.38** | **17.57** | N/A | N/A | N/A | N/A |
| heart03 | **1.78** | **3.85** | **3.21** | **2.91** | **8.42** | **7.08** | **9.47** | **8.96** | **5.34** | N/A | N/A |
| heart04 | -0.18 | **4.82** | **3.39** | **2.68** | **4.49** | **4.55** | **2.17** | **5.71** | **6.08** | N/A | N/A |
| heart05 | **0.42** | **1.78** | **3.61** | **5.66** | **5.23** | **6.27** | **9.69** | **1.68** | -1.16 | **6.91** | N/A |
| heart06 | **0.03** | **2.01** | **2.20** | **2.43** | **5.83** | **4.70** | **5.47** | **9.65** | **4.26** | **8.83** | N/A |
| heart07 | -0.10 | **1.58** | **2.72** | **3.56** | **5.85** | **5.26** | **4.92** | **4.59** | **7.90** | **3.53** | **2.75** |

Table 7.3: Percentage difference in $W^{\max}$ between NVOL and VOL for the Heart dataset across different $k$. Each value is the result of $(W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100$. Cells marked with N/A were not run.

Table 7.3 shows the results for the Heart dataset. The first thing to notice is that NVOL improves $W^{\max}$ in almost all runs. On average, NVOL achieves a 4.70% lower $W^{\max}$ than VOL across all runs, with improvements lying mostly between 2% and 7% and peaking at 17.57%. Table 7.3 records in total 4 losses, averaging 0.69% across them with a peak at 1.30%. As was already apparent from Table 7.1, these graphs are very similar in structure: they have comparable standard deviations and a very close average number of edges per vertex.

This similarity is also reflected in the partitioning results, as most values within each column are relatively close to one another, with only a few exceptions. Looking at the average value per graph confirms this too. Table 7.4 shows the average percentage difference of $W^{\max}$ for the Heart dataset. All seven averages fall within a narrow 4.15% range, from heart04 at 3.75% to heart02 at 7.90%, reinforcing that the graphs behave similarly under partitioning.

| **Graphs** | **Avg. difference** | **Best result** | **Worst result** |
|---|---|---|---|
| heart01 | 4.13% | 14.40% | −1.30% |
| heart02 | 7.90% | 17.57% | 0.79% |
| heart03 | 5.67% | 9.47% | 1.78% |
| heart04 | 3.75% | 6.08% | −0.18% |
| heart05 | 4.01% | 9.69% | −1.16% |
| heart06 | 4.54% | 9.65% | 0.03% |
| heart07 | 3.87% | 7.90% | −0.10% |

Table 7.4: Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL (Heart dataset).

Notably, NVOL generally performs better for higher $k$ values. On average, NVOL performs two to three times as many moves as VOL during a single refinement round, and the number of partitions clearly amplifies this effect. This allows NVOL to explore more options and increases the chance of finding a better partitioning. Additionally, since more partitions lead to more moves, they also reduce the likelihood of getting stuck in a local minimum. This does, however, greatly affect the time NVOL takes to partition, which averages between 7 and 11 times slower than VOL across the Heart dataset.

The runs where NVOL loses are: heart01 with $k = 2$, heart04 with $k = 2$, heart05 with $k = 512$, and heart07 with $k = 2$. Almost all of the failures occur when $k = 2$. A likely cause is that, at each refinement step, NVOL sets a ceiling on the outgoing volume of a partition and then allows any move in which the affected partitions do not exceed this maximum. When the number of partitions is large, it is easier to find a move that decreases this value. However, with only two partitions, it is more likely that no available move can improve it. NVOL therefore tends to get stuck in a local minimum, which may result in a higher $W^{\max}$ than VOL.

Interestingly, Figure 7.1 reveals that in every run with $k = 2$, the standard deviation for NVOL is 0. This means that both partitions produced by NVOL have exactly the same outgoing volume. By only allowing moves that maintain or decrease $W^{\max}$, possibly at the cost of partitions with lower outgoing volumes, NVOL drives the partitions closer together. The values in Figure 7.1 have been normalized using the average outgoing volume of the partitions.

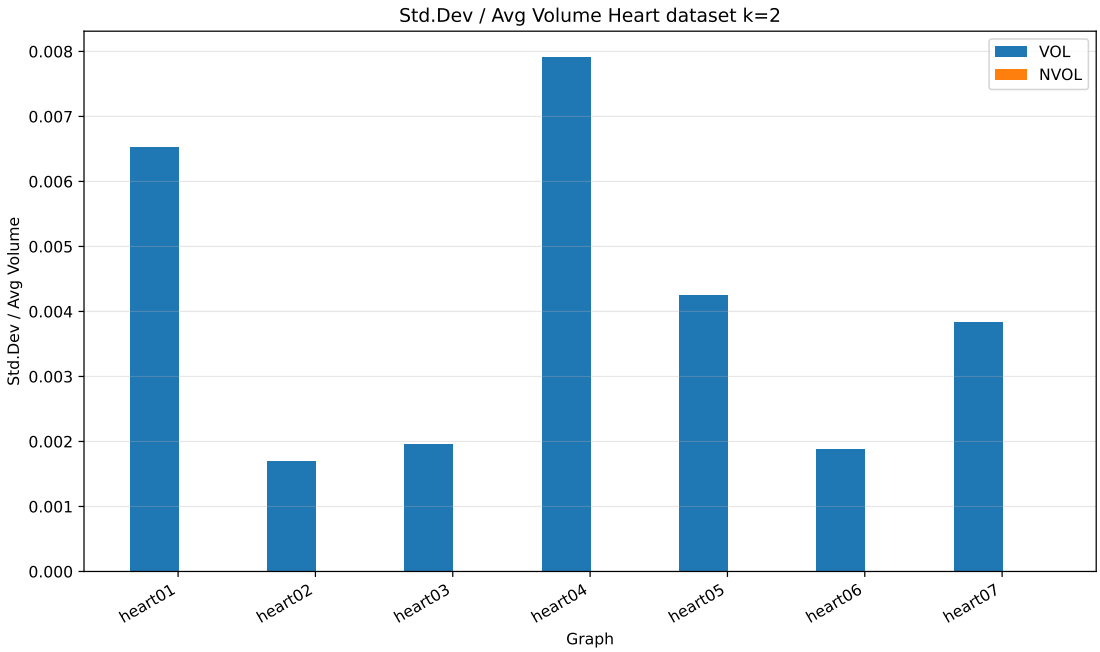


Figure 7.1: Standard deviation normalized by the average volume for runs with $k = 2$ on the Heart dataset. VOL (blue) shows nonzero variation across graphs; NVOL (orange) is 0 on every graph, meaning its two partitions have identical outgoing volume.

NVOL achieves a slightly lower standard deviation than VOL in almost all runs, indicating a more even distribution of outgoing volume across partitions. All the partitioned graphs in the Heart dataset seem to follow the same pattern where the standard deviation peaks at $k = 8$ and then gradually decreases as the number of

partitions grows, regardless of objective function. NVOL maintains an edge throughout, generally having the best difference for $k = 2$ and $k = 4$ runs. Table 7.5 presents the standard deviations for VOL and NVOL on heart04, which sits in the middle of the dataset and is representative of this behavior; the other graphs show similar numbers. Both objective functions produce similar distributions of values, but NVOL consistently

| **Partitions** | **VOL std. dev.** | **NVOL std. dev.** | **VOL tot. vol.** | **NVOL tot. vol.** |
|---|---|---|---|---|
| **2** | 34.5 | 0.0 | 8,717 | 8,802 |
| **4** | 529.48 | 410.57 | 17,241 | 17,143 |
| **8** | 856.11 | 832.52 | 26,791 | 26,805 |
| **16** | 670.39 | 652.02 | 40,485 | 40,383 |
| **32** | 491.23 | 488.32 | 60,323 | 59,879 |
| **64** | 340.87 | 338.55 | 88,753 | 88,676 |
| **128** | 238.4 | 231.87 | 128,651 | 127,155 |
| **256** | 162.38 | 157.77 | 184,231 | 183,157 |
| **512** | 110.09 | 105.71 | 257,257 | 255,394 |

Table 7.5: Comparison of the standard deviation of the outgoing volumes and total volume on heart04 graph for VOL and NVOL

yields lower standard deviation. This is because NVOL favors moves that bring partition volumes closer together, whereas VOL's purely greedy approach optimizes only for volume reduction. As a result, NVOL's partition volumes are more evenly balanced.

Table 7.5 also reveals that NVOL produces a lower total communication volume in many runs. This is not just the case for heart04, but across the entire Heart dataset. On heart04, NVOL loses twice ($k = 2$ and $k = 8$) on total communication volume. When it wins, the improvement is small, but notable: NVOL beats VOL on the very metric VOL optimizes for. Given the regular structure of the heart graphs, it appears that the better-balanced partitions NVOL produces tend to indirectly reduce total communication volume as well.

This increased balance does come with some trade-offs. Figure 7.2 shows a plot of the outgoing volumes of heart04 when partitioned with $k = 4$. The blue bars represent VOL's partitions outgoing volumes and the orange ones NVOL. By looking closely at each bar one might notice that NVOL's bars are all higher than VOL's apart from the tallest one. This makes sense as NVOL moves the vertices in a way that pushes down the maximum potentially at the cost of other partitions. As a result the pattern shown in Figure 7.2 can be found in most results from NVOL.

Aside from $k = 2$ cases discussed earlier, heart05 with $k = 512$ is the only graph–partition combination in the Heart dataset where NVOL produces a higher $W^{\max}$ than VOL. The exact cause is hard to pin down from the data alone; NVOL probably made some moves from which no further improvement could be found. However, with some simple tuning (Section 7.3), NVOL recovers a win on this case, suggesting it had indeed settled into a local minimum it could not escape. Even in this failure, the loss is narrow: NVOL exceeds $W^{\max}_{\text{VOL}}$ by only 1.16%. It does, however, produce 1% lower total communication volume and a lower standard deviation than VOL.

Throughout the Heart dataset, NVOL has the best $W^{\max}$ in 57 out of 61 runs (4.70% on average, peaking at 17.57%) and produces more evenly balanced partition volumes in almost every run. It also yields slightly lower total communication volume in many cases, despite not optimizing for it directly. The main weakness is at $k = 2$, where the

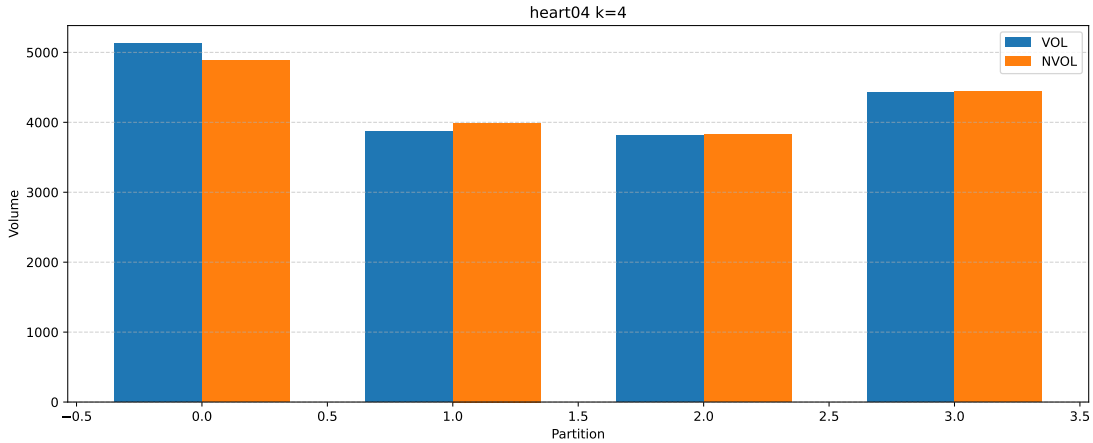


Figure 7.2: Comparison of the outgoing communication volumes of the partitions of heart04 $k = 4$ for VOL (blue) and NVOL (orange).

constrained move space limits NVOL's ability to lower $W^{\max}$. NVOL does, however, produce perfectly even partitions in these cases.

### 7.2.2 SuiteSparse Matrix Collection

| **Graphs** | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 | 2048 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| pa2010 | -5.02 | **6.69** | **10.11** | **3.08** | **12.54** | **11.03** | **15.92** | **9.46** | **16.03** | N/A | N/A |
| coPapersDBLP | -32.09 | -17.81 | -8.82 | **1.78** | **0.41** | **24.10** | **40.37** | **48.44** | **56.70** | N/A | N/A |
| packing -500x100x100 -b050 | **3.61** | **0.02** | **4.42** | **5.11** | **8.49** | **5.76** | **9.61** | **13.64** | **9.37** | **11.97** | N/A |
| channel -500x100x100 -b050 | **10.67** | **8.79** | **10.32** | **9.83** | **9.92** | **13.69** | **11.57** | **9.82** | **8.54** | **9.72** | **3.16** |
| hugetric-00000 | **9.02** | **12.71** | **12.24** | **14.62** | **12.94** | **7.63** | **16.01** | **18.53** | **1.76** | **16.23** | **19.72** |
| italy_osm | **0.00** | -1.02 | **0.90** | -1.09 | **8.87** | **4.00** | **3.85** | -0.94 | -21.21 | -6.82 | **2.86** |
| hugetric-00020 | **7.50** | **7.05** | **7.47** | **7.27** | **8.39** | **8.05** | **14.48** | **10.97** | **13.83** | **10.95** | **20.52** |
| rgg_n_2_23_s0 | -6.46 | **4.47** | **6.86** | **6.33** | **4.84** | **6.47** | -0.64 | **8.16** | **18.60** | **7.30** | **0.0** |
| delaunay_n23 | -1.48 | **3.19** | **3.35** | **4.53** | **3.12** | **8.54** | **6.98** | **6.47** | **8.33** | **6.05** | **5.88** |
| hugetrace-00020 | **8.27** | **8.34** | **7.07** | **7.41** | **8.51** | **8.46** | **11.12** | **10.38** | **13.29** | **12.95** | **15.63** |
| delaunay_n24 | -2.60 | **4.71** | **5.97** | **2.40** | **6.23** | **5.93** | **4.85** | **7.98** | **5.01** | **2.93** | **5.01** |
| kmer_V2a | -0.28 | **10.34** | **20.27** | **3.98** | **1.53** | **11.14** | -4.49 | **7.94** | **18.98** | **18.00** | N/R |
| kmer_U1a | **0.72** | **6.71** | **14.18** | **26.55** | **29.76** | **35.88** | **28.85** | **46.34** | **34.06** | **37.64** | N/R |

Table 7.6: Percentage difference in $W^{\max}$ between NVOL and VOL for the SuiteSparse Matrix Collection selection across different $k$. Each value is the result of $(W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100$. Cells marked with N/R and N/A were not run.

Now consider the results for the SuiteSparse Matrix collection in Table 7.6. The graphs vary considerably in size and structure, and so do the results. Regardless, NVOL is able to achieve a lower $W^{\max}$ than VOL 89% of the time. Compared to the Heart dataset, NVOL wins more convincingly with an average improvement of 9.19% across all runs. NVOL is able to produce an improvement above 20% on several runs with a peak at 56.70% for coPapersDBLP with $k = 512$. However, NVOL loses more often and by larger margins than on the Heart dataset. In the 15 recorded losses NVOL averages 7.40% loss with a peak at 32.09% for coPapersDBLP with $k = 2$. Most losses are concentrated in the italy_osm graph and for several runs with $k = 2$.

Table 7.7 reports the average per graph, where once again the graph difference is represented by the spread. Apart from italy_osm, NVOL wins on average by at least 4.34% and on the biggest graph, kmer_U1a, it wins on average by 26.07%. The table, however, does not seem to suggest any obvious relationship between graph properties (size, average degree, or degree variance) and the magnitude of the improvement, as the two next best winners are very different from each other.

| **Graphs** | **Avg. difference** | **Best result** | **Worst result** |
|---|---:|---:|---:|
| pa2010 | 8.87% | 16.03% | −5.02% |
| coPapersDBLP | 12.56% | 56.7% | −32.09% |
| packing-500x100x100-b050 | 7.20% | 13.64% | 0.02% |
| channel-500x100x100-b050 | 9.64% | 13.69% | 3.16% |
| hugetric-00000 | 12.86% | 19.72% | 1.76% |
| italy_osm | −0.96% | 8.87% | −21.21% |
| hugetric-00020 | 10.59% | 20.52% | 7.05% |
| rgg_n_2_23_s0 | 5.59% | 18.60% | −6.46% |
| delaunay_n23 | 5.00% | 8.54% | −1.48% |
| hugetrace-00020 | 10.13% | 15.63% | 7.07% |
| delaunay_n24 | 4.34% | 7.98% | −2.60% |
| kmer_V2a | 8.74% | 20.27% | −4.49% |
| kmer_U1a | 26.07% | 46.34% | 0.72% |

Table 7.7: Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL (SuiteSparse Matrix Collection).

| **Graphs** | **VOL std. dev.** | **NVOL std. dev.** | **VOL tot. vol.** | **NVOL tot. vol.** |
|---|---:|---:|---:|---:|
| pa2010 | 6.5 | 0.0 | 425 | 460 |
| coPapersDBLP | 8,854.0 | 5.5 | 166,524 | 243,349 |
| rgg_n_2_23_s0 | 36.0 | 0.0 | 8,684 | 9,322 |
| delaunay_n23 | 2.0 | 0.0 | 5,672 | 5,760 |
| delaunay_n24 | 5.0 | 0.0 | 8,302 | 8,528 |
| kmer_V2a | 2,876.5 | 0.0 | 265,323 | 271,834 |

Table 7.8: Comparison of the standard deviation of the outgoing volumes and total volume of VOL and NVOL for $k = 2$ runs where NVOL is worse in Table 7.6.

In total, NVOL loses 15 of 136 runs. Setting italy_osm aside (discussed below), this drops to 10 losses, 6 of which occur at $k = 2$. As with the Heart dataset, NVOL achieves a more even volume distribution at the cost of a higher per-partition maximum on these specific $k = 2$ runs. Because the standard deviations vary across orders of magnitude for VOL, we report them in Table 7.8 rather than plot them as in Figure 7.1. The table clearly shows how VOL creates disparity between the partitions with extremes like coPapersDBLP where it has a standard deviation 8,848.5 above NVOL's.

As opposed to the Heart dataset, NVOL is unable to improve on the total communication volume for the runs in Table 7.8. Most of the results are close to each other, except coPapersDBLP where NVOL's partitioning total volume is 1.5 times greater than VOL. This does however seem reasonable given that coPapersDBLP has a very high degree variance, meaning that the amount of edges each vertex has is very

different from vertex to vertex. This can explain why even when perfectly balanced, the high degree variance greatly affects the communication volume.

In general, NVOL achieves a lower standard deviation than VOL across all graphs. The picture is more mixed for total communication volume: both kmer graphs and coPapersDBLP show visibly higher total volumes under NVOL, since NVOL promotes moves that increase the per-partition average if they decrease the per-partition maximum. On channel-500x100x100-b050, packing-500x100x100-b050, hugetric, and hugetrace, on the other hand, NVOL yields a lower total communication volume. This is interesting, since these are also the graphs where NVOL achieves the most consistent improvements on $W^{\max}$. The remaining graphs produce total volumes close to VOL's, either slightly higher or slightly lower.

Looking at Table 7.6, we see that structurally similar graphs produce similar results, just as in the Heart dataset. The different structural properties of these graphs are reflected in how well NVOL manages to lower $W^{\max}$. Of all the graph types tested, NVOL struggles mostly with italy_osm, the only road network in the dataset. For the others it works consistently well, lowering $W^{\max}$ and distributing the volume more evenly than VOL across partitions.

### italy_osm graph

NVOL struggles greatly with the italy_osm graph, achieving its second worst result across all our runs of $-21.21\%$ at $k = 512$ and failing to improve on VOL's standard deviation in most runs. The italy_osm graph represents the Italian road network extracted from OpenStreetMap, with vertices placed at intersections and edges corresponding to the road segments between them. Road networks differ structurally from the regular meshes of the Heart dataset in several important ways. They are very sparse, with vertex degrees rarely exceeding 5. More importantly, they are strongly heterogeneous: dense urban clusters are connected by long, thin stretches of rural roads, and natural features such as rivers, mountains, and coastlines act as small separators between regions of very different sizes.

This heterogeneity makes it noticeably harder for NVOL to push down $W^{\max}$ while keeping partitions balanced. A partition containing a dense urban area cannot easily be matched in volume to one covering a sparse rural region, so the moves NVOL relies on to equalize partition volumes have limited room to operate. The low average degree compounds the issue, since each move affects only a handful of edges and therefore shifts the outgoing volume by very little, leaving NVOL with little leverage to escape configurations where the maximum cannot be lowered.

Despite the negative overall picture, NVOL is not uniformly worse on this graph: wins and losses occur in roughly equal numbers, with a best result of $8.87\%$ at $k = 32$. The losses are simply larger in magnitude than the wins, which drags the average into negative territory. Figure 7.3 makes this asymmetry visible, a handful of moderate wins offset by a dramatic $-21.21\%$ loss at $k = 512$. This suggests NVOL can occasionally find a favorable configuration even on irregular structures, but the sparse and heterogeneous nature of road networks does not seem to benefit from its partition-aware optimization; if anything, it works against it.

### coPapersDBLP graph

The graph yielding the best and worst result is coPapersDBLP in Table 7.6. coPapersDBLP is a citation network derived from the DBLP computer science

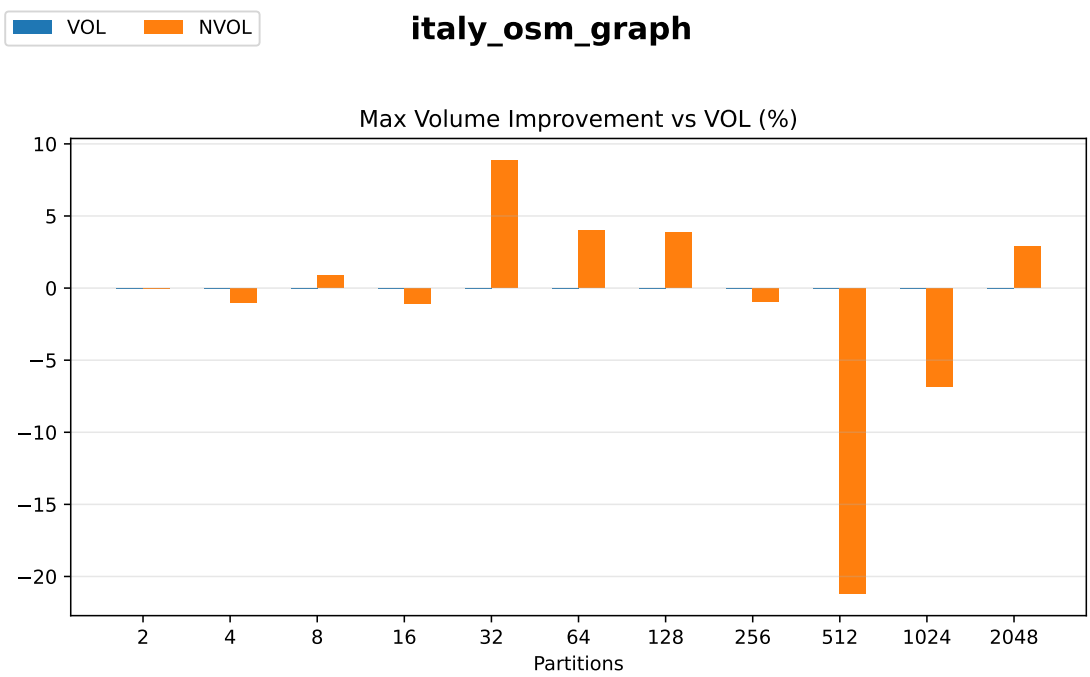


Figure 7.3: Percentage difference in $W^{\max}$ between NVOL and VOL for italy_osm graph across different $k$. Each value is the result of $(W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100$.

bibliography. Citation networks have a different structure from the meshes and road networks discussed earlier. They are scale-free, with a heavy-tailed degree distribution: a small number of hub vertices carry a disproportionate share of the connections, while a long tail of vertices have only a handful. This is reflected in coPapersDBLP itself, where the maximum degree of 3,299 sits nearly two orders of magnitude above the average degree of 56. Citation networks are also small-world, with short average path lengths despite the network's size, and exhibit strong community structure corresponding to research areas or fields. This means that the average shortest distance between any two vertices grows very slowly with the graph size.

The results seem to follow a trend where they improve as the number of partitions increases. Despite the 3 negative results, the positive ones outweigh them, giving the graph an average difference of 12.56%, the third highest in Table 7.7. NVOL produces much more balanced results here as well. Table 7.8 already showed that at $k = 2$ NVOL achieves a standard deviation of 5.5 against VOL's 8,854.

These large differences persist for $4 \leq k \leq 32$. Figure 7.4 shows the standard deviations across all coPapersDBLP runs. Comparing this with the percentage differences in Table 7.6 suggests a plausible explanation. The heavy-tailed degree distribution of citation networks makes it difficult to produce a balanced, low-volume result when partitions are few. A handful of hub vertices end up dominating their host partition's outgoing volume, and without many options for moves NVOL is unable to redistribute this volume efficiently. In fact, as the number of partitions increases, it is easier to find optimal moves for such heavy hub vertices, in turn making the heavy-tailed structure easier to exploit. This does, however, cost a higher variance in the partition volumes.

Figure 7.5 shows an example that supports this hypothesis. At $k = 8$, NVOL achieves an almost perfectly balanced distribution with a standard deviation of 0.33 against

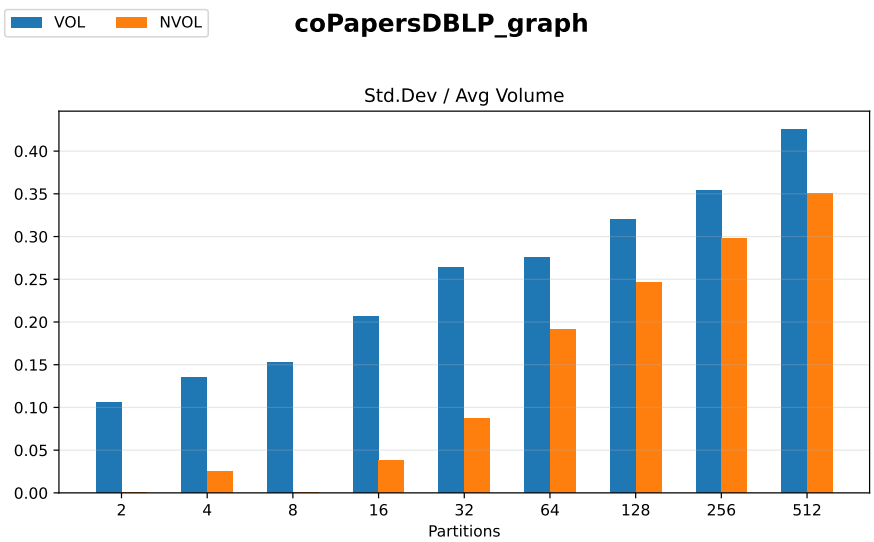


Figure 7.4: Comparison of the average normalized standard deviation on coPapersDBLP across different $k$ for VOL (blue) and NVOL (orange).

VOL's 10,473. As with the heart04 case discussed earlier, this balance comes at the cost of pushing each partition's volume closer to each other. NVOL does not manage to improve $W^{\max}$, losing by 8.82% to VOL, and as is visible in the figure, all the partitions are pushed closer to each other.

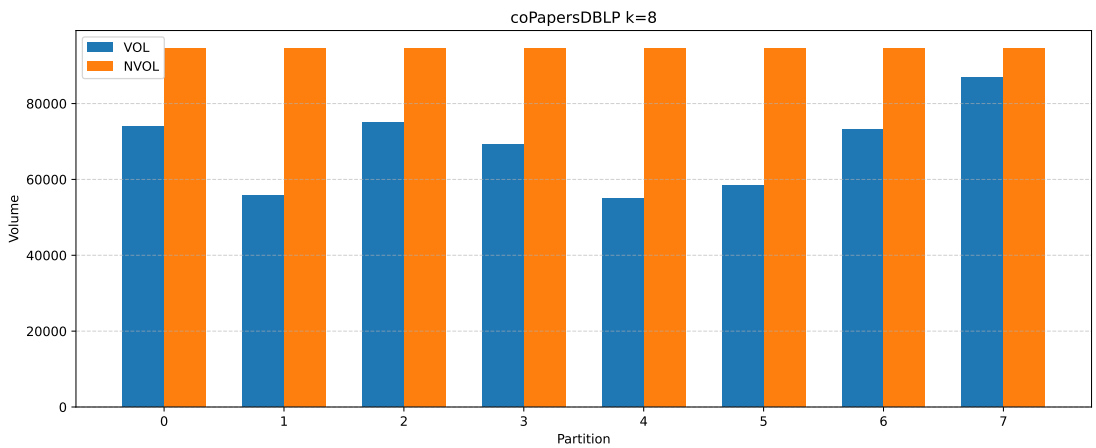


Figure 7.5: Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 8$ for VOL (blue) and NVOL (orange).

At high partition counts, NVOL's advantage on coPapersDBLP becomes substantial: $W^{\max}$ improves by 40.37%, 48.44%, and 56.70% over VOL at $k = 128$, $k = 256$, and $k = 512$ respectively. Figure 7.6 illustrates the $k = 128$ case. NVOL's partition volumes cluster tightly around the mean while still achieving a significantly lower $W^{\max}$ than VOL. The struggle at low $k$ and these gains at high $k$ are two sides of the same coin: the heavy-tailed degree distribution that prevents balancing with few partitions becomes a feature NVOL can exploit once $k$ is large enough to spread the hubs across many partitions. Citation networks therefore appear to be a graph class where NVOL's balancing strategy has real potential, provided $k$ is sufficiently high.

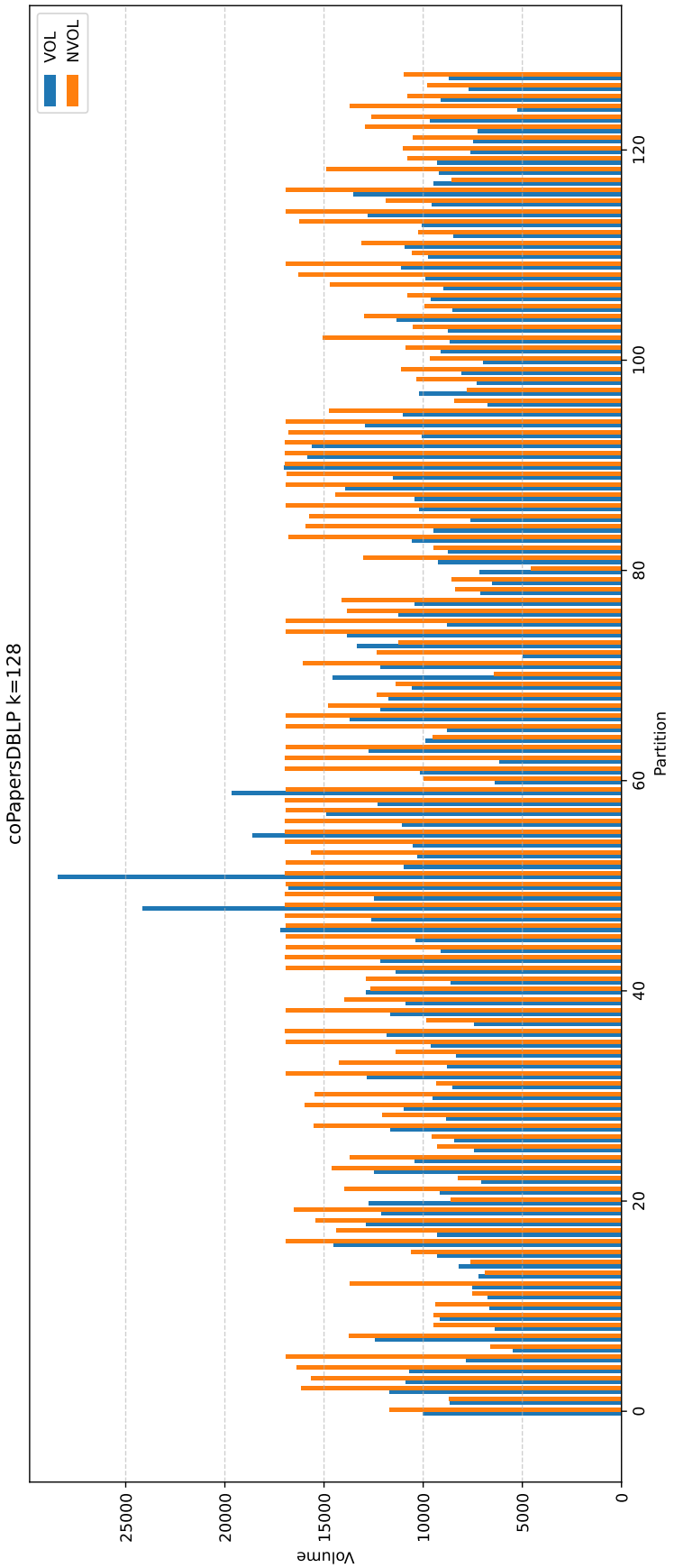


Figure 7.6: Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 128$ for VOL (blue) and NVOL (orange).

### kmer_U1a and kmer_V2a graphs

Both kmer_U1a and kmer_V2a are protein k-mer graphs from the GenBank group of the SuiteSparse Matrix Collection, originally distributed as part of the 2018 GraphChallenge. Each vertex represents a short segment of amino acids (a k-mer) extracted from protein sequences, and edges connect k-mers that overlap in the underlying sequence. These are the two largest graphs in our selection, with kmer_U1a containing 67.7 million vertices and kmer_V2a 55 million. Both instances are extremely sparse, with an average degree of approximately 2, and highly fragmented into many small connected components. These graphs are the only ones marked N/R: NVOL was unable to finish partitioning them at $k = 2048$ due to their size. We discuss this limitation of NVOL compared to VOL, and ideas for addressing it, in Section 8.1.

Despite their structural similarity, the two graphs yield noticeably different partitioning results. On the larger instance, kmer_U1a, NVOL achieves consistent improvements over VOL on $W^{\max}$, averaging 26.07% across all values of $k$. On kmer_V2a the gain is more modest: NVOL loses to VOL at $k = 2$ and $k = 128$, but still averages a positive 8.74%. On the total volume metric, however, both graphs fail to improve over VOL, and the gap on kmer_V2a widens as $k$ grows.

The two graphs also differ in how evenly NVOL distributes volume across partitions. On kmer_V2a, NVOL produces a higher run-to-run standard deviation than on kmer_U1a, where the standard deviation remains zero for all $k < 16$ and only begins to grow from $k = 16$ onward. Figures 7.7 and 7.8 show the total volume and standard deviation across all runs on kmer_V2a and kmer_U1a respectively.

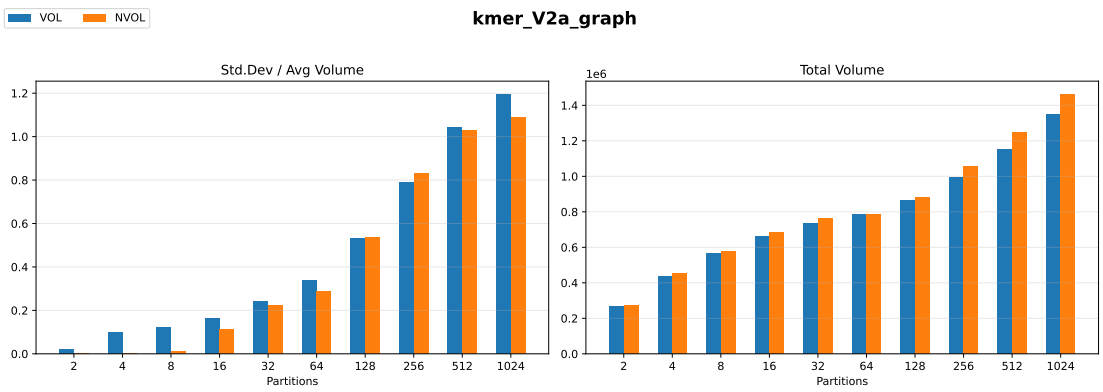


Figure 7.7: Comparison of the average normalized standard deviation and total communication volume on kmer_V2a across different $k$ for VOL (blue) and NVOL (orange).

A possible explanation for these differences lies in the disparity of fragmentation. Although the two graphs differ in size by only 18%, kmer_U1a contains 44,132 connected components against 15,343 in kmer_V2a, almost three times as many. The much larger number of small components in kmer_U1a may give the partitioner more flexibility when distributing volume across partitions, while in kmer_V2a the components are on average larger and harder to distribute as $k$ grows, which is consistent with the higher standard deviation observed for both VOL and NVOL compared to kmer_U1a.

The combination of low standard deviation and improvement on $W^{\max}$ is most evident in the high $k$ runs of kmer_U1a. Figure 7.9 shows the per-partition outgoing-volume distribution of VOL and NVOL at $k = 128$. As in Figure 7.6, NVOL produces partitions with much more uniform volumes than VOL and visibly reduces $W^{\max}$. The

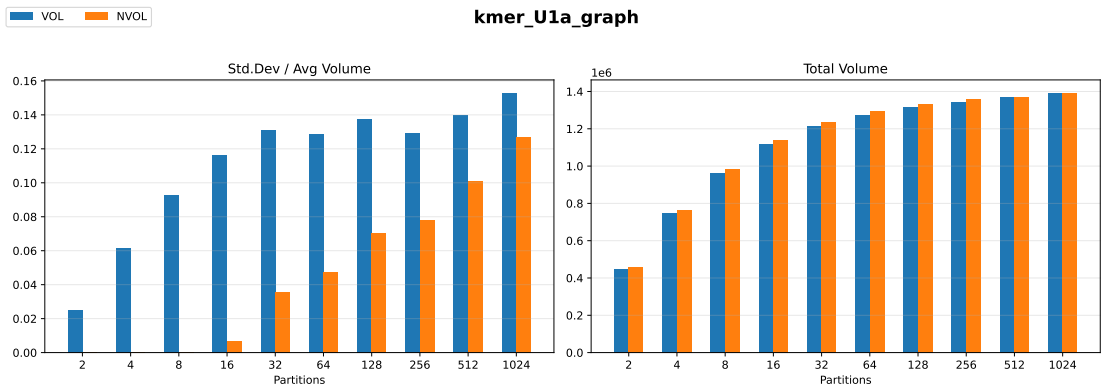


Figure 7.8: Comparison of the average normalized standard deviation and total communication volume on kmer_U1a across different $k$ for VOL (blue) and NVOL (orange).

pattern becomes even more pronounced at higher values of $k$, but those plots are harder to read, so we show only $k = 128$ here.

Overall, the kmer graphs paint a nuanced picture of NVOL's behavior on very sparse, highly fragmented inputs. On $W^{\max}$, NVOL clearly outperforms VOL on average for both graphs. On the total volume, however, neither graph shows an improvement, indicating that NVOL's redistribution of load comes at the cost of a higher average volume per partition. The contrast between the two instances further suggests that fragmentation level matters: kmer_U1a, with nearly three times as many connected components, admits more stable and more consistent improvements, whereas the larger components of kmer_V2a constrain how the volume can be balanced as $k$ grows. This uneven volume distribution is visible for both NVOL and VOL.

### Numerical simulation graphs

The most represented group of graphs in our selection are numerical simulation graphs. These graphs are derived from physical domains, a chunk of fluid, a 3D box of space, a mechanical structure, by discretizing the domain into a mesh of small cells. Each vertex represents a cell, and edges connect cells that share a face or boundary in the mesh. This type of graph tends to be very regular: most vertices have a similar number of neighbors, and there are no hub vertices. Five of our graphs fall into this category: channel-500x100x100-b050, packing-500x100x100-b050, hugetric-00000, hugetric-00020, and hugetrace-00020.

NVOL performs consistently well on these graphs, improving $W^{\max}$ in every single run (55 runs). As mentioned earlier, NVOL also beats VOL on total communication volume in several of these cases. Structurally these graphs are close cousins of the Heart dataset: both classes are originally regular meshes with low degree variance, which partly explains why NVOL performs so well. The symmetry in these graphs lets NVOL easily find moves that improve the volume, as opposed to some of the previous examples.

Table 7.9 shows the percentage difference in total volume and standard deviation for hugetric-00000, the numerical simulation graph with the highest average improvement on $W^{\max}$. We chose this graph specifically because it is representative of the class. A positive value indicates that NVOL achieves a lower value than VOL, following the same convention as Table 7.6. NVOL achieves a lower total volume in all runs, with an average improvement of 10.59%. NVOL also produces a consistently better standard deviation, suggesting that the volumes are more evenly distributed across the partitions.

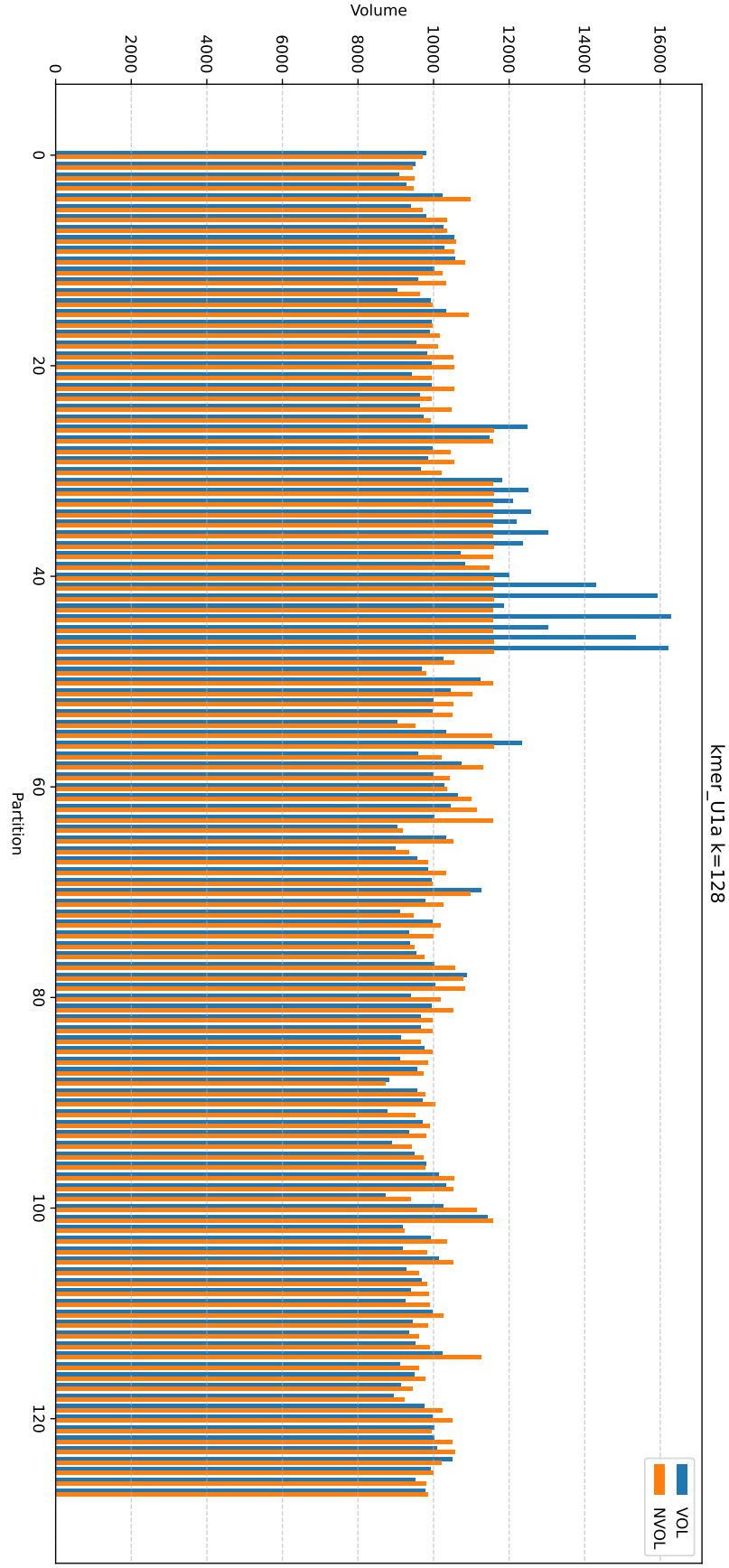


Figure 7.9: Comparison of the outgoing communication volumes of the partitions of kmer_U1a $k = 128$ for VOL (blue) and NVOL (orange).

| Partitions | Percentage difference tot. vol. | Percentage difference std. dev. |
|---|---|---|
| **2** | 9.02% | 0.00% |
| **4** | 12.78% | 12.85% |
| **8** | 12.36% | 9.46% |
| **16** | 13.00% | 16.67% |
| **32** | 11.94% | 14.47% |
| **64** | 9.89% | 4.41% |
| **128** | 10.12% | 14.73% |
| **256** | 9.39% | 14.72% |
| **512** | 8.85% | 14.59% |
| **1024** | 10.77% | 20.28% |
| **2048** | 8.39% | 18.09% |

Table 7.9: Comparison of the percentage difference of total volume and standard deviation on hugetric-00000 for VOL and NVOL.

On hugetric-00000, NVOL produces excellent partitions, achieving a lower total communication volume, a lower $W^{\max}$, and a more even distribution than VOL. This pattern is also visible across the remaining numerical simulation graphs, making NVOL a valuable partitioning objective for this graph class.

### Geometric graphs

The remaining graphs all fall under the geometric graph class. These graphs are derived from points in space rather than from physical adjacency or relationships. pa2010, for example, represents the Pennsylvania Census 2010 redistricting graph, with vertices as Census blocks and edges connecting blocks that share a border. The two delaunay graphs are synthetic Delaunay triangulations of random points in a 2D unit square, while rgg_n_2_23_s0 is a random geometric graph. All four have low degree variance with no hub vertices.

Because these graphs differ slightly from each other, they do not follow as clean a pattern as the numerical simulation graphs. They do, however, produce broadly similar averages in Table 7.7. Figure 7.10 shows a plot of the values in Table 7.6. The three synthetic graphs average around 4-6%, while pa2010 sits notably above at nearly 9%.

pa2010 is structurally similar to italy_osm, yet NVOL performs well on it. The likely reason is that pa2010 covers a single US state, whose census blocks are far more uniform in size and density than the urban-rural-mountain extremes spanning all of Italy. Its average degree of 4 with low variance reflects this uniformity, giving NVOL the kind of regular structure it can balance effectively.

Both the best and worst results in this group come from rgg_n_2_23_s0, which records a loss of 6.46% at $k = 2$ and a win of 18.60% at $k = 512$. It is also the only graph with a loss outside $k = 2$, losing 0.64% at $k = 128$. As with heart05 at $k = 512$, there is no clear indication of why NVOL loses on this run. The partitions still have a lower standard deviation than VOL, and their total volumes are very close. This appears to be once again a case of a succession of bad moves leading NVOL into a configuration it cannot escape.

Geometric graphs are therefore a class where NVOL is reliably useful: it consistently lowers $W^{\max}$, produces total volumes very close to VOL, and yields a slightly lower standard deviation in most runs.

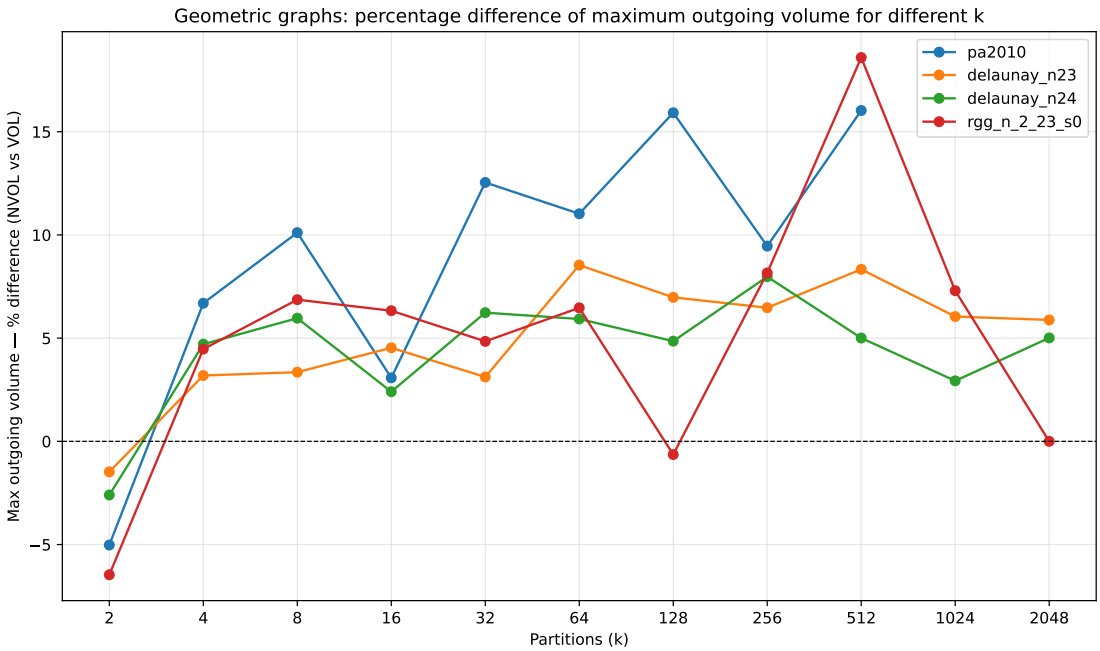


Figure 7.10: Percentage difference in $W^{\max}$ between NVOL and VOL for our selection of geometric graphs across different $k$ $((W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100)$.

## 7.3 Tuning NVOL

METIS comes with the option to further tune your refinement. By default the algorithm runs with an imbalance factor of 3%, however, this number can be defined at runtime. Increasing this value allows greater imbalance in the partition weights, which in turn should allow moves that were not possible under the default imbalance. We test NVOL at a 4% imbalance, one percentage point above the default. Although small, this does affect the resulting partitions, and we wanted to see if it helps NVOL to perform better than VOL. We compare NVOL with a 4% imbalance to the same VOL setup, 3% imbalance, as the previous sections. The goal of this subsection is to show how NVOL is affected by this change.

### 7.3.1 Heart dataset with 4% imbalance

Table 7.10 shows the percentage difference between VOL and NVOL. Compared to Table 7.3, it is immediately clear that NVOL loses more often to VOL. In Table 7.3, NVOL loses 4 of 61 runs, as opposed to 11 of 61 here. It does, however, win a few more times by 10% or more. In a similar manner, the losses are also more significant, with five of them at 4% or above.

In Table 7.11 we compare the average results of each graph. For several graphs, the larger wins pull the average above default NVOL despite the additional losses. NVOL with 4% works best on average on smaller graphs, as the average difference decreases as the graph size grows. This can possibly be explained by the fact that in smaller graphs, some moves are more restricted by the weight constraints. A move that decreases $W^{\max}$ may not be accepted because the partition weight is too high. It is also visible in heart01, heart03, and heart04 that NVOL with 4% produces the best result.

This imbalance comes at a trade-off: more uncertainty in how much NVOL loses.

| Graphs | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 | 2048 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| heart01 | -1.30 | **2.08** | **16.49** | **5.19** | **18.18** | N/A | N/A | N/A | N/A | N/A | N/A |
| heart02 | **0.57** | **2.92** | **3.41** | **12.55** | **4.62** | **6.76** | **15.45** | N/A | N/A | N/A | N/A |
| heart03 | -1.46 | -4.52 | **2.37** | **4.08** | **6.68** | **6.27** | **18.01** | **8.08** | **11.07** | N/A | N/A |
| heart04 | -4.99 | **9.60** | **17.20** | **4.19** | **1.84** | **5.65** | **4.20** | **7.35** | **6.89** | N/A | N/A |
| heart05 | **5.34** | -1.05 | **3.11** | **6.64** | **8.75** | **7.24** | **8.93** | **0.00** | **5.75** | **2.60** | N/A |
| heart06 | **2.75** | -2.86 | **4.51** | -7.14 | **0.35** | **9.89** | **9.77** | **2.82** | -0.93 | **9.16** | N/A |
| heart07 | -0.86 | **4.79** | **2.75** | **2.83** | **4.12** | -4.42 | **7.61** | **7.44** | **7.45** | -6.97 | **5.40** |

Table 7.10: Percentage difference in $W^{\max}$ between NVOL 4% and VOL for the Heart dataset across different $k$. Each value is the result of $((W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/A were not run.

The default run's worst loss was only $-1.30\%$, while nine of the losses here are larger in magnitude. This is very visible in heart06 and heart07, each registering three losses and dragging their averages below the default NVOL configuration. Total volume and standard deviation are relatively close to each other in most runs for NVOL and NVOL with 4% imbalance.

| Graphs | NVOL avg. diff. | NVOL 4% avg. diff. | NVOL 4% Best result | NVOL 4% Worst result |
|---|---|---|---|---|
| heart01 | 4.13% | 8.13% | 18.18% | −1.30% |
| heart02 | 7.90% | 6.61% | 15.45% | 0.57% |
| heart03 | 5.67% | 5.62% | 18.01% | −4.52% |
| heart04 | 3.75% | 5.77% | 17.20% | −4.99% |
| heart05 | 4.01% | 4.73% | 8.93% | −1.05% |
| heart06 | 4.54% | 2.83% | 9.89% | −7.14% |
| heart07 | 3.87% | 2.74% | 7.61% | −6.97% |

Table 7.11: Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL 4% (Heart dataset).

Interestingly, NVOL recovers an earlier loss at heart05 with $k = 512$ here. With the 4% imbalance, NVOL is able to perform 11,888 more moves compared to the default run. The looser restriction allows NVOL to escape the configuration it got stuck in during the previous run, reaching a better result. This is the only case in the Heart dataset where the imbalance recovers an earlier loss.

On the Heart dataset, raising the imbalance to 4% helps small graphs and hurts large ones. However, even in small graphs, the losses are larger and more frequent than under the default imbalance. In the next section we will examine whether this pattern holds for the diverse SuiteSparse graphs.

### 7.3.2 SuiteSparse Matrix Collection with 4% imbalance

Table 7.12 shows the results on the selection of the SuiteSparse graphs when running NVOL with 4% imbalance. As with the Heart dataset, the spread of values widens in both directions. Runs with an NVOL improvement of 20% or more rise from 13 to 18, while the number of losses also grows from 15 to 18, with an average loss of $-14.54\%$. The 4% imbalance produces two new extremes as well: a peak of 57.51% for coPapersDBLP at $k = 512$ and a low of $-59.78\%$ for italy_osm at $k = 16$.

| Graphs | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 | 2048 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| pa2010 | -18.26 | **11.85** | **13.48** | **3.69** | **3.14** | **14.07** | **12.44** | **11.49** | **19.08** | N/A | N/A |
| coPapersDBLP | -31.82 | -17.75 | -9.85 | **1.09** | **10.57** | **22.58** | **40.18** | **49.04** | **57.51** | N/A | N/A |
| packing -500x100x100 -b050 | -9.51 | -0.76 | -7.29 | **11.83** | **12.33** | **5.62** | **5.46** | **14.52** | **3.51** | **11.76** | N/A |
| channel -500x100x100 -b050 | **11.49** | **9.46** | **9.48** | **9.22** | **17.57** | **12.91** | **10.12** | **10.60** | -2.73 | **9.92** | **5.18** |
| hugetric-00000 | **16.01** | **14.29** | **6.68** | **15.72** | **13.17** | **4.04** | **8.69** | **16.14** | **12.94** | **12.08** | **21.60** |
| italy_osm | **4.41** | -6.12 | -32.43 | -59.78 | **26.61** | **2.00** | **13.08** | **1.89** | **12.12** | -25.00 | -22.86 |
| hugetric-00020 | **4.81** | **11.31** | **17.91** | **1.62** | **1.02** | **0.94** | **15.03** | **11.90** | **21.36** | **13.43** | **13.10** |
| rgg_n_2_23_s0 | -4.55 | **3.40** | **2.00** | **9.06** | **5.45** | **7.47** | **8.25** | **3.54** | **16.22** | **3.17** | **5.01** |
| delaunay_n23 | -1.94 | **5.67** | **3.07** | **6.61** | **4.88** | **4.24** | **8.32** | **11.99** | **10.83** | **7.67** | **6.47** |
| hugetrace-00020 | **3.83** | **8.04** | **3.70** | **3.57** | **12.26** | **5.74** | **0.97** | **13.59** | **12.94** | **10.23** | **24.78** |
| delaunay_n24 | **0.77** | **5.34** | **10.34** | **6.33** | **9.57** | **1.41** | **3.82** | **7.80** | **2.98** | -6.84 | **1.37** |
| kmer_V2a | -2.14 | **9.35** | **20.52** | **13.63** | **0.66** | **6.48** | -2.14 | **6.54** | **23.22** | **21.48** | N/R |
| kmer_U1a | **0.53** | **6.59** | **14.77** | **26.60** | **29.53** | **35.46** | **29.07** | **49.69** | **33.61** | **34.63** | N/R |

Table 7.12: Percentage difference in $W^{\max}$ between NVOL 4% and VOL for the SuiteSparse Matrix Collection selection across different $k$. Each value is the result of $((W^{\max}_{\text{VOL}} - W^{\max}_{\text{NVOL}})/W^{\max}_{\text{VOL}} \times 100)$. Cells marked with N/R and N/A were not run.

Comparing the per-graph averages in Table 7.13, most rows shift very little, typically within $\pm 1\%$. The clearest exception is italy_osm, whose average drops from $-0.96\%$ to $-7.83\%$. Best-case results improve on nearly every graph, usually by 1-3%, with italy_osm and hugetrace-00020 showing the largest single-run gains. The worst-case results, by contrast, deteriorate almost everywhere; only three graphs see an improvement at their worst tail: hugetric-00000, rgg_n_2_23_s0, and kmer_V2a. The greatest deterioration occurs on italy_osm, whose worst result collapses from $-21.21\%$ to $-59.78\%$. As in the Heart dataset, total volume and standard deviation differ only slightly between the two configurations.

| Graphs | NVOL avg. diff. | NVOL 4% avg. diff. | NVOL 4% Best result | NVOL 4% Worst result |
|---|---|---|---|---|
| pa2010 | 8.87% | 7.89% | 19.08% | −18.26% |
| coPapersDBLP | 12.56% | 13.51% | 57.51% | −31.82% |
| packing-500x100x100-b050 | 7.20% | 4.75% | 14.52% | −9.51% |
| channel-500x100x100-b050 | 9.64% | 9.38% | 17.57% | −2.73% |
| hugetric-00000 | 12.86% | 12.85% | 21.60% | 4.04% |
| italy_osm | −0.96% | −7.83% | 26.61% | −59.78% |
| hugetric-00020 | 10.59% | 10.22% | 21.36% | 0.94% |
| rgg_n_2_23_s0 | 5.59% | 5.37% | 16.22% | −4.55% |
| delaunay_n23 | 5.00% | 6.16% | 11.99% | −1.94% |
| hugetrace-00020 | 10.13% | 9.06% | 24.78% | 0.97% |
| delaunay_n24 | 4.34% | 3.90% | 10.34% | −6.84% |
| kmer_V2a | 8.74% | 9.76% | 23.22% | −2.14% |
| kmer_U1a | 26.07% | 26.05% | 49.69% | 0.53% |

Table 7.13: Average, best, and worst percentage difference in $W^{\max}$ per graph for VOL and NVOL 4% (SuiteSparse Matrix Collection).

Here too, NVOL manages to recover some earlier losses. The most striking case is italy_osm at $k = 512$, which moves from $-21.21\%$ to $12.12\%$. In total, four earlier losses

are recovered: italy_osm at $k = 256$ and $k = 512$, rgg_n_2_23_s0 at $k = 128$, and delaunay_n24 at $k = 2$. On the other hand, seven previously winning results regress into losses, slightly fewer than the eight regressions in the Heart dataset.

Unlike in the Heart dataset, graph size does not appear to drive the difference between the two configurations here. The 4% imbalance mainly stretches the tails, pushing the best results higher and the worst results lower, while leaving the per-graph averages largely unchanged. NVOL also continues to struggle on the road-network structure of italy_osm, where the loosened constraint produces even larger negative results.

The 4% setting is therefore best treated as a per-graph tuning option rather than a default: it can yield better results in some cases, but with no clear predictor of when it helps and when it hurts.

## 7.4 Model estimates

Thus far we have evaluated how NVOL performs based on outgoing communication volume metrics. However, the end goal of graph partitioning is to improve performance when running the partitions. In order to test if the lower volumes actually improved performance we used the model estimate described in Section 6.2. We point out that this is simply a model, and not actual runtime, meaning that these values are an approximation that closely models a real machine.

We test our graphs for a setup with 32 processors with a startup latency and a set of bandwidths we got sent by the authors of [45]. These values are taken from the bandwidths of an actual parallel computer and are not a universal set. The results would look different using another machine. Unfortunately, we were not able to retrieve the bandwidth for a bigger machine in order to test some of our best results at higher $k$.

Since a parallel application is generally as slow as its slowest process, we start this evaluation by looking at this value first. On each graph we compare the slowest process runtime on VOL and NVOL and calculate the percentage difference. If we define the runtime as $R$, each value in the following Table 7.14 equals $\Delta R_{\text{NVOL}} = (R_{\text{VOL}} - R_{\text{NVOL}})/R_{\text{VOL}}$, where positive values indicate NVOL is faster and vice versa. We check this value for both NVOL and NVOL with 4% imbalance to default VOL, all at $k = 32$.

Table 7.14 shows how across both configurations, NVOL wins 60% of the time. More specifically, using the default imbalance it has 13 wins, one tie and six losses, while the 4% imbalance runs have 11 wins and nine losses. NVOL with 4% maintains the same pattern (see Section 7.2.2) of having a greater variance in its results with a greater span between best and worst results. Both approaches struggle greatly for coPapersDBLP yielding slower runtimes. The Heart dataset and the numerical simulations deliver good results. For default NVOL they are all positive, NVOL with 4% records a few losses here too.

Figure 7.11 shows the best result out of the bigger graphs. On heart06, which has 4 million vertices, NVOL manages to improve the slowest time by 6.6%. We choose to focus on this graph instead of heart01, which has only 1,340 vertices. In a small graph like heart01, the volume affects the runtime less than the startup latency does.

| **Graphs** | $\Delta R_{\text{NVOL}}$ | $\Delta R_{\text{NVOL4\%}}$ |
|---|---|---|
| heart01 | 0.0% | 9.99% |
| heart02 | 0.6% | 0.2% |
| heart03 | 1.1% | -8.6% |
| heart04 | −0.1% | 6.3% |
| heart05 | 0.4% | 5.6% |
| heart06 | 6.6% | 6.6% |
| heart07 | 3.2% | 4.3% |
| pa2010 | 0.2% | −0.3% |
| coPapersDBLP | −29.4% | −17.97% |
| packing-500x100x100-b050 | 6.5% | 4.6% |
| channel-500x100x100-b050 | 5.6% | 5.7% |
| hugetric-00000 | 2.8% | 3.6% |
| italy_osm | 0.1% | −27.8% |
| hugetric-00020 | 2.0% | −8.8% |
| rgg_n_2_23_s0 | −1.0% | −1.3% |
| delaunay_n23 | −0.9% | −9.3% |
| hugetrace-00020 | 2.6% | −5.7% |
| delaunay_n24 | −6.1% | 0.1% |
| kmer_V2a | −3.7% | −2.9% |
| kmer_U1a | 2.2% | 2.1% |

Table 7.14: Percentage difference for the slowest runtime of NVOL and NVOL 4% compared to VOL (e.g. $\Delta R_{\text{NVOL}} = (R_{\text{VOL}} - R_{\text{NVOL}})/R_{\text{VOL}}$).

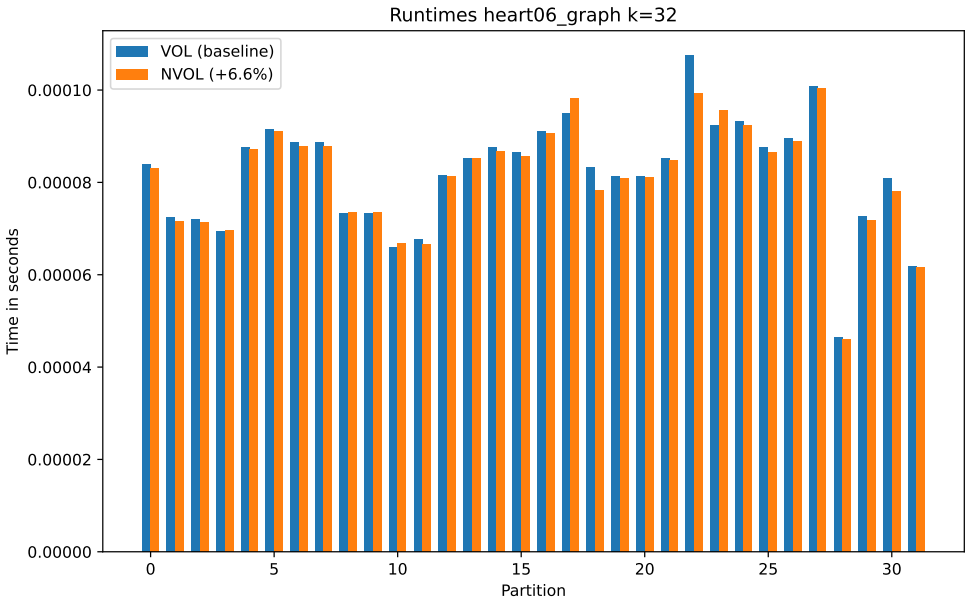


Figure 7.11: Estimated runtimes per partition on heart06 $k = 32$ for VOL and NVOL. The value after NVOL represents the percentage difference between the highest value for VOL and NVOL.

The next subsections will discuss the two cases where NVOL produces significantly worse runtimes. As it will become clear, the runtime is not only dependent on the

single maximum value, but also on other factors like distribution of volume and which partitions have the highest number of neighbors.

### 7.4.1 coPapersDBLP graph estimate results

Compared to VOL, NVOL and NVOL with 4% imbalance produce 29.4% and 17.97% slower runtimes respectively. Both have lower max outgoing volumes per partition, NVOL by 0.41% and NVOL with 4% by 10.57%. Both yield also a lower maximum incoming volume, improving on VOL by 1.37% and 22.19% respectively.

Although NVOL achieves a lower maximum per-partition volume (incoming and outgoing), its overall distribution is shifted upward: both the mean and the lighter partitions are larger than in VOL. Using the model estimate described in Section 6.2, lighter partitions finish first and free up bandwidth for the larger partitions. VOL's many small partitions provide this relief, whereas NVOL's denser distribution prolongs the contention. As a result the heaviest partitions need to wait longer regardless of their own volume. Figures 7.13 and 7.12 show the clear difference in how the volume is distributed between VOL and NVOL. To make this distinction as clear as possible, we chose to display the volumes of NVOL with 4% as it shows this behavior best.

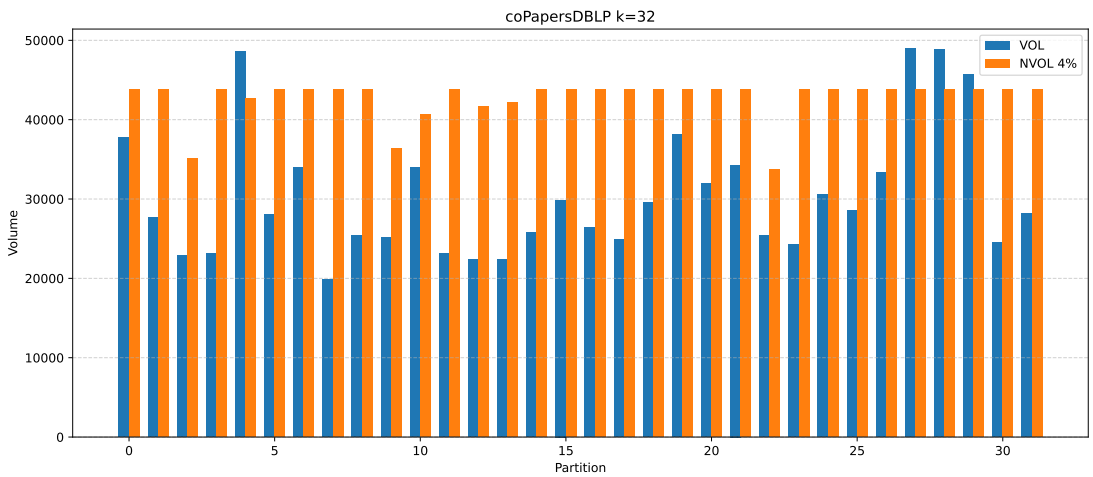


Figure 7.12: Comparison of the outgoing communication volumes of the partitions of coPapersDBLP $k = 32$ for VOL (blue) and NVOL (orange) with 4% imbalance.

### 7.4.2 italy_osm graph estimate results

At first glance the results for this graph seem wrong. NVOL improves the max outgoing volume by 8.87% and NVOL with 4% imbalance by 26.61%, yet their runtimes change by 0.1% and −27.8% respectively. The 4% variant essentially loses on runtime by roughly the same margin it gains on volume. To understand why, we need to look beyond the maximum outgoing and incoming volumes.

The estimates are produced using Equation 6.2, which gives the runtime for partition $i$ as the time needed to send and receive its messages, plus the startup latency for each outgoing message. Table 7.15 reports the relevant statistics for VOL and NVOL 4% on italy_osm with $k = 32$. NVOL has lower values on every metric except the maximum number of neighboring partitions.

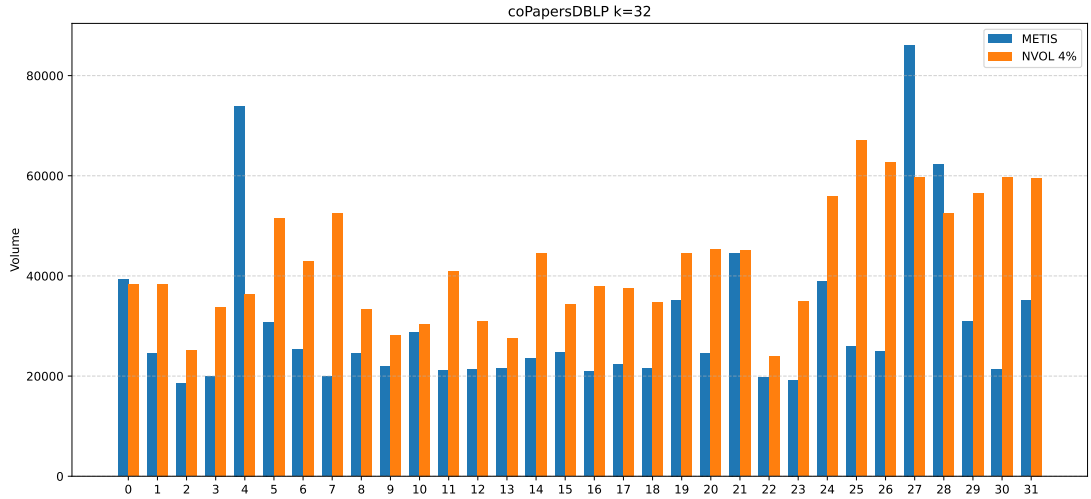


Figure 7.13: Comparison of the incoming communication volumes of the partitions of coPapersDBLP $k = 32$ for VOL (blue) and NVOL (orange) with 4% imbalance.

| Objective function | Max outgoing vol. | Max incoming vol. | Max neighbor partitions |
|---|---|---|---|
| VOL | 124 | 123 | 7 |
| NVOL 4% | 91 | 96 | 9 |

Table 7.15: Communication statistics for VOL and NVOL 4% on italy_osm with $k = 32$.

Both partitionings have very low volumes overall, likely due to the sparse structure of road network graphs. This is reflected directly in the estimate: the largest $t^{\text{recv}}$ value is $6.9 \cdot 10^{-7}$ for VOL and $5.58 \cdot 10^{-7}$ for NVOL. So far, the volume advantage of NVOL behaves as expected.

The latency term, however, tells a different story. We use a latency constant of $3.8\,\mu s$, which by Equation 6.2 is multiplied by the number of neighboring partitions. This constant is already an order of magnitude larger than $t^{\text{recv}}$, so once it is scaled by the neighbor count it dominates the estimate. NVOL has one partition with nine neighbors and two with eight, all above VOL's maximum of seven, and these three partitions stand out as the slowest in Figure 7.14. Instead, Figure 7.15 shows what the times would look like if we ignore the latency. Here it is clear that NVOL's wins on volume bring better times.

This example illustrates that on graphs with very low communication volumes, latency can dominate the runtime estimate, and a partitioning that improves volume at the cost of more neighbors per partition may end up slower overall. The specific numbers here are tied to our setup, but the broader pattern, that latency matters more in low-volume graphs, is general.

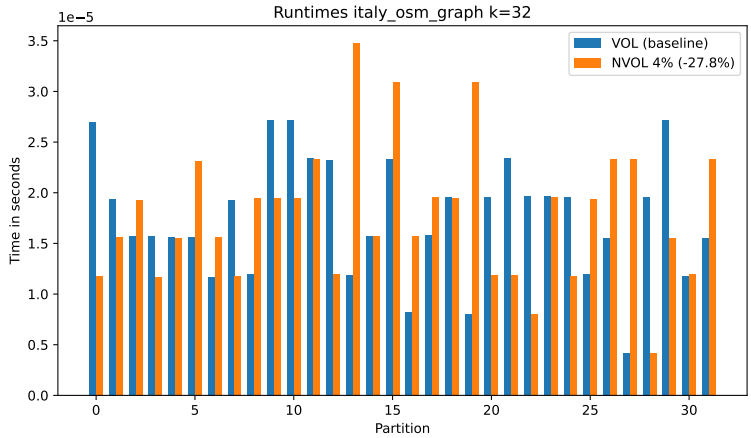


Figure 7.14: Estimated runtimes per partition on italy_osm $k = 32$ for VOL (blue) and NVOL 4% (orange). The value after NVOL is the percentage difference between the highest runtime for VOL and for NVOL.

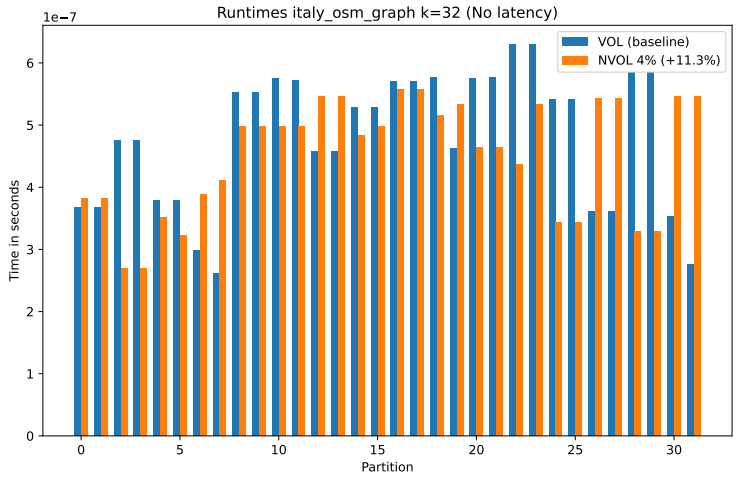


Figure 7.15: Estimated runtimes per partition on italy_osm $k = 32$ for VOL (blue) and NVOL 4% (orange) when ignoring the latency. The value after NVOL is the percentage difference between the highest runtime for VOL and for NVOL.

# Chapter 8

# Future Work

This chapter outlines areas where the work in this thesis could be improved, along with related projects that would extend it.

## 8.1 Improved efficiency

One of METIS's main strengths is its partitioning speed, and the additional structures and operations introduced by NVOL inevitably affect this, particularly on larger graphs. For small graphs the partitioning time is essentially unchanged, but on larger ones NVOL was observed to be significantly slower than METIS running with the total communication volume objective.

The overhead has a straightforward explanation: NVOL adds code to METIS's refinement stage and increases the number of iterations performed there. METIS itself is carefully designed to avoid redundant work, most of the volume operations are performed during graph projection. This way the refinement stage can reuse it rather than reiterating over the graph. NVOL breaks this pattern by recomputing the gain information for each vertex it visits during refinement, effectively repeating work that METIS already performs at projection time.

Three considerations led us not to integrate NVOL's gain calculation directly into the projection stage. First, NVOL has a larger memory requirement than total communication volume. As described in Chapter 4, total communication volume requires a vector whose length equals the number of partitions a vertex neighbors, whereas NVOL requires a table. For a vertex with a gain vector of length $k$, the corresponding gain table has dimensions $(k + 1) \times k$, so a gain vector of length 5 already implies a $6 \times 5$ table per vertex. Second, METIS uses incremental updates between moves to keep the volume gains of affected vertices current; implementing the equivalent logic for NVOL was outside the scope of this thesis. Third, and perhaps most decisive, time constraints during development, combined with the considerable effort required to address the first two points, prevented us from pursuing this integration further.

This decision was nonetheless preceded by an attempt at the integration. Our strategy allocated the exact memory needed for each gain table and deallocated it once the corresponding vertex left the boundary; gain updates were handled by simply recomputing all neighbors of the moved vertex. In principle this is much closer to what METIS does and should have improved performance. In practice, however, the algorithm became even slower than the original NVOL implementation, with the frequent allocation and deallocation of memory as the dominant cost. A better solution would be to allocate a single block of memory at projection time, sized from an estimate of how

many boundary vertices the finer graph will contain, with additional logic to extend the block when the estimate falls short and to release it once refinement completes. Combined with proper incremental updates of neighboring vertices' gains after each move, this would allow NVOL to partition in a time frame close to METIS, at the cost of higher memory use.

## 8.2 Introduce hardware knowledge

## 8.3 More testing

# Chapter 9

# Conclusions

# Bibliography


[1] Stephen T. Barnard and Horst D. Simon. 'Fast multilevel implementation of recursive spectral bisection for partitioning unstructured problems'. In: *Concurrency: Practice and Experience* 6.2 (1994), pp. 101–117. DOI: https://doi-org.ezproxy.uio.no/10.1002/cpe.4330060203.

[2] Erik G. Boman, Karen D. Devine and Sivasankaran Rajamanickam. 'Scalable matrix computations on large scale-free graphs using 2D graph partitioning'. In: *SC '13: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 2013, pp. 1–12. DOI: 10.1145/2503210.2503293.

[3] Aydın Buluç, Kamesh Madduri, Henning Meyerhenke, Peter Sanders, David A. Bader and Dorothea Wagner. 'Graph partitioning for scalable distributed graph computations'. In: *Graph Partitioning and Graph Clustering*. Vol. 588. Contemporary Mathematics. American Mathematical Society, 2013, pp. 83–101. DOI: 10.1090/conm/588/11709.

[4] Aydın Buluç, Henning Meyerhenke, Ilya Safro, Peter Sanders and Christian Schulz. 'Recent Advances in Graph Partitioning'. In: *Algorithm Engineering: Selected Results and Surveys*. Springer International Publishing, 2016, pp. 117–158. DOI: 10.1007/978-3-319-49487-6_4. URL: https://doi.org/10.1007/978-3-319-49487-6_4.

[5] U.V. Catalyurek and C. Aykanat. 'Hypergraph-partitioning-based decomposition for parallel sparse-matrix vector multiplication'. In: *IEEE Transactions on Parallel and Distributed Systems* 10.7 (1999), pp. 673–693. DOI: 10.1109/71.780863.

[6] Ümi˙t V. Çatalyürek, Cevdet Aykanat and Bora Uçar. 'On Two-Dimensional Sparse Matrix Partitioning: Models, Methods, and a Recipe'. In: *SIAM Journal on Scientific Computing* 32.2 (2010), pp. 656–683. DOI: 10.1137/080737770.

[7] Ümit Çatalyürek et al. 'More Recent Advances in (Hyper)Graph Partitioning'. In: 55.12 (2023). DOI: 10.1145/3571808.

[8] Ümit V. Çatalyürek, Mehmet Deveci, Kamer Kaya, Bora Uçar, Henning Meyerhenke, Peter Sanders, David A. Bader and Dorothea Wagner. 'UMPa: A multi-objective, multi-level partitioner for communication minimization'. In: *Graph Partitioning and Graph Clustering*. Vol. 588. Contemporary Mathematics. American Mathematical Society, 2013, pp. 53–65. DOI: 10.1090/conm/588/11704.

[9] P.K. Chan, M.D.F. Schlag and J.Y. Zien. 'Spectral K-way ratio-cut partitioning and clustering'. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 13.9 (1994), pp. 1088–1096. DOI: 10.1109/43.310898.

[10] Jingsheng Jason. Cong and Joseph R. Shinnerl. *Multilevel Optimization in VLSICAD*. 1st ed. 2003. Combinatorial Optimization. Springer US, 2003. DOI: 10.1007/978-1-4757-3748-6.

[11] Timothy A. Davis and Yifan Hu. 'The university of Florida sparse matrix collection'. In: *ACM Trans. Math. Softw.* 38.1 (Dec. 2011). DOI: 10.1145/2049662.2049663. URL: https://sparse.tamu.edu/.

[12] Ralf Diekmann, Robert Preis, Frank Schlimbach and Chris Walshaw. 'Shape-optimized mesh partitioning and load balancing for parallel adaptive FEM'. In: *Parallel Computing* 26.12 (2000). Graph Partitioning and Parallel Computing, pp. 1555–1581. ISSN: 0167-8191. DOI: https://doi.org/10.1016/S0167-8191(00)00043-0.

[13] Executive Office of the President. *Executive Order 13702—Creating a National Strategic Computing Initiative*. Federal Register, Vol. 80, No. 149, pp. 46177–46180. July 2015. URL: https://www.federalregister.gov/documents/2015/08/03/2015-19183/creating-a-national-strategic-computing-initiative (visited on 13/04/2026).

[14] C.M. Fiduccia and R.M. Mattheyses. 'A Linear-Time Heuristic for Improving Network Partitions'. In: *19th Design Automation Conference*. 1982, pp. 175–181. DOI: 10.1109/DAC.1982.1585498.

[15] Miroslav Fiedler. 'A property of eigenvectors of nonnegative symmetric matrices and its application to graph theory'. In: *Czechoslovak mathematical journal* 24.4 (1975), pp. 619–633. DOI: 10.21136/cmj.1975.101357.

[16] Santo Fortunato. 'Community detection in graphs'. In: *Physics Reports* 486.3 (2010), pp. 75–174. DOI: https://doi.org/10.1016/j.physrep.2009.11.002.

[17] M.R. Garey, D.S. Johnson and L. Stockmeyer. 'Some simplified NP-complete graph problems'. In: *Theoretical Computer Science* 1.3 (1976), pp. 237–267. DOI: https://doi.org/10.1016/0304-3975(76)90059-1.

[18] Bruce Hendrickson. 'Graph partitioning and parallel solvers: Has the emperor no clothes?' In: *Solving Irregularly Structured Problems in Parallel*. Springer Berlin Heidelberg, 1998, pp. 218–225. DOI: 10.1007/BFb0018541.

[19] Bruce Hendrickson and Tamara G Kolda. 'Graph partitioning models for parallel computing'. In: *Parallel Computing* 26.12 (2000). Graph Partitioning and Parallel Computing, pp. 1519–1534. DOI: https://doi.org/10.1016/S0167-8191(00)00048-X.

[20] Bruce Hendrickson and Robert Leland. 'A multilevel algorithm for partitioning graphs'. In: *Proceedings of the 1995 ACM/IEEE Conference on Supercomputing*. Supercomputing '95. ACM, 1995, pp. 28–42. DOI: 10.1145/224170.224228.

[21] Bruce Hendrickson and Robert Leland. 'An Improved Spectral Graph Partitioning Algorithm for Mapping Parallel Computations'. In: *SIAM Journal on Scientific Computing* 16.2 (1995), pp. 452–469. DOI: 10.1137/0916028.

[22] John L. Hennessy and David A. Patterson. 'A new golden age for computer architecture'. In: *Commununications of the ACM* 62.2 (2019), pp. 48–60. DOI: 10.1145/3282307.

[23] Costas Iliopoulos, Jason T. L Wang, Mourad Elloumi and Albert Y Zomaya. 'Wiley Series on Bioinformatics: Computational Techniques and Engineering'. In: *Pattern Recognition in Computational Molecular Biology*. John Wiley & Sons, Incorporated, 2015. DOI: 10.1002/9781119078845.scard.

[24] Andrew B. Kahng, Jens. Lienig, Igor L. Markov and Jin. Hu. *VLSI Physical Design: From Graph Partitioning to Timing Closure*. 1st ed. 2011. Springer Netherlands, 2011. DOI: 10.1007/978-90-481-9591-6.

[25] G. Karypis and V. Kumar. 'Analysis of Multilevel Graph Partitioning'. In: *Supercomputing '95:Proceedings of the 1995 ACM/IEEE Conference on Supercomputing*. 1995, pp. 29–29. DOI: 10.1145/224170.224229.

[26] George Karypis and Vipin Kumar. 'A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs'. In: *SIAM Journal on Scientific Computing* 20.1 (1998), pp. 359–392. DOI: 10.1137/S1064827595287997.

[27] George Karypis and Vipin Kumar. 'Multilevel k-way Partitioning Scheme for Irregular Graphs'. In: *Journal of Parallel and Distributed Computing* 48.1 (1998), pp. 96–129. DOI: https://doi.org/10.1006/jpdc.1997.1404.

[28] George Karypis and Vipin Kumar. 'Parallel Multilevel series k-Way Partitioning Scheme for Irregular Graphs'. In: *SIAM Review* 41.2 (1999), pp. 278–300. DOI: 10.1137/S0036144598334138.

[29] George Karypis and Vipin Kumar. 'Parallel Multilevel series k-Way Partitioning Scheme for Irregular Graphs'. In: *SIAM Review* 41.2 (1999), pp. 278–300. DOI: 10.1137/S0036144598334138.

[30] B. W. Kernighan and S. Lin. 'An efficient heuristic procedure for partitioning graphs'. In: *The Bell System Technical Journal* 49.2 (1970), pp. 291–307. DOI: 10.1002/j.1538-7305.1970.tb01770.x.

[31] Johannes Langguth, Sebastian Schlag and Christian Schulz. 'Load-Balanced Bottleneck Objectives in Process Mapping'. In: (2020). DOI: 10.48550/arXiv.2001.09645.

[32] Hao Li, G.W. Rosenwald, J. Jung and Chen-ching Liu. 'Strategic Power Infrastructure Defense'. In: *Proceedings of the IEEE* 93.5 (2005), pp. 918–933. DOI: 10.1109/JPROC.2005.847260.

[33] Juan Li and Chen-Ching Liu. 'Power system reconfiguration based on multilevel graph partitioning'. In: *2009 IEEE Bucharest Power Tech*. IEEE, 2009, pp. 1–5. DOI: 10.1109/PTC.2009.5281971.

[34] H Martinez-Navarro, B Rodriguez, A Bueno-Orovio and A Minchole. *Repository for modelling acute myocardial ischemia: simulation scripts and torso-heart mesh*. 2019. URL: https://ora.ox.ac.uk/objects/uuid:951b086c-c4ba-41ef-b967-c2106d87ee06.

[35] Irene Moulitsas, George Karypis, Anu G Bourgeois and Si Quing Zheng. 'Architecture Aware Partitioning Algorithms'. In: *Algorithms and Architectures for Parallel Processing*. Vol. 5022. Lecture Notes in Computer Science. Springer Berlin / Heidelberg, 2008, pp. 42–53. DOI: 10.1007/978-3-540-69501-1_6.

[36] M. E. J. Newman. 'Community detection and graph partitioning'. In: *Europhysics Letters* 103.2 (2013). DOI: 10.1209/0295-5075/103/28003.

[37] M. E. J. Newman. *Networks: An Introduction*. Oxford University Press, 2010. DOI: 10.1093/acprof:oso/9780199206650.001.0001.

[38] Joel Nishimura and Johan Ugander. 'Restreaming graph partitioning: simple versatile algorithms for advanced balancing'. In: *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '13. ACM, 2013, pp. 1106–1114. DOI: 10.1145/2487575.2487696.

[39] Fareed Qararyah, Mohamed Wahib, Doğa Dikbayır, Mehmet Esat Belviranli and Didem Unat. 'A computational-graph partitioning method for training memory-constrained DNNs'. In: *Parallel Computing* 104-105 (2021), pp. 1–12. DOI: https://doi.org/10.1016/j.parco.2021.102792.

[40] Semih Salihoglu, Jennifer Widom, Magdalena Balazinska, Alex Szalay, Ahmet Sacan, Tamas Budavari and Alexandra Meliou. 'GPS: a graph processing system'. In: *Proceedings of the 25th International Conference on Scientific and Statistical Database Management*. ACM, 2013, pp. 1–12. DOI: 10.1145/2484838.2484843.

[41] Horst D. Simon and Shang-Hua Teng. 'How Good is Recursive Bisection?' In: *SIAM Journal on Scientific Computing* 18.5 (1997), pp. 1436–1445. DOI: 10.1137/S1064827593255135.

[42] Xianfeng Song, Yi Zou and Zheng Shi. 'CaPGNN: Optimizing parallel graph neural network training with joint caching and resource-aware graph partitioning'. In: *Neurocomputing (Amsterdam)* 675 (2026), pp. 1–15. DOI: 10.1016/j.neucom.2026.132978.

[43] Isabelle Stanton and Gabriel Kliot. 'Streaming graph partitioning for large distributed graphs'. In: *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '12. ACM, 2012, pp. 1222–1230. DOI: 10.1145/2339530.2339722.

[44] Herb Sutter and James Larus. 'Software and the concurrency revolution'. In: *ACM Queue* 3.7 (2005), pp. 54–62. DOI: 10.1145/1095408.1095421.

[45] Andreas Thune, Sven-Arne Reinemo, Tor Skeie and Xing Cai. 'Detailed Modeling of Heterogeneous and Contention-Constrained Point-to-Point MPI Communication'. In: *IEEE Transactions on Parallel and Distributed Systems* 34.5 (2023), pp. 1580–1593. DOI: 10.1109/TPDS.2023.3253881.

[46] Charalampos Tsourakakis, Christos Gkantsidis, Bozidar Radunovic and Milan Vojnovic. 'FENNEL: streaming graph partitioning for massive scale graphs'. In: *Proceedings of the 7th ACM International Conference on Web Search and Data Mining*. WSDM '14. ACM, 2014, pp. 333–342. DOI: 10.1145/2556195.2556213.

[47] C. Walshaw, M. Cross and M.G. Everett. 'Parallel Dynamic Graph Partitioning for Adaptive Unstructured Meshes'. In: *Journal of Parallel and Distributed Computing* 47.2 (1997), pp. 102–108. DOI: https://doi.org/10.1006/jpdc.1997.1407.

[48] David P. Williamson and David B. Shmoys. 'An Introduction to Approximation Algorithms'. In: *The Design of Approximation Algorithms*. Cambridge University Press, 2011, pp. 3–26.